PuckVision - Entwicklung eines Eishockeytrainingssystem



Diplomarbeit verfasst von

Iason Vasiliadis
Fabian Gassler

Betreuer: Prof. Dr. techn. Bernhard Prommegger

HTBLuVA Salzburg

Abteilung für Elektronik und Technische Informatik

HTBLuVA Salzburg

Abgabevermerk:

Salzburg, 2 April 2025 übernommen von:

Eidesstattliche Erklärung

Wir erklären an Eides statt, dass wir die vorliegende Arbeit selbstständig verfasst, andere als die angegebenen Quellen/Hilfsmittel nicht benutzt, und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche kenntlich gemacht haben.

02.04.2025

Datum

Iason Vasiliadis

Fabian Gassler

Diplomarbeit Dokumentation

Namen der Verfasser/innen	Iason Vasiliadis (Projektleiter) Fabian Gassler
Jahrgang / Schuljahr	5AHEL, 2024/2025
Thema der Diplomarbeit	PuckVision - Entwicklung eines Eishockeytrainingssystem
Kooperationspartner	-
Aufgabenstellung	Eishockeytraining ist aufwändig und teuer- PuckVision bietet eine komfortable Alternative. Das Trainingsmodul ermöglicht das Verbessern des Stick-Handlings abseits des Eises. Die Möglichkeit eines schnellen Aufbaus und eines platzarmen Verstauens machen es noch attraktiver. Verschiedene Modi ermöglichen stätig neue Herausforderungen.
Realisierung	Unter der Spieloberfläche, die aus einer Plexiglasscheibe besteht, befinden sich mehrere Messplatinen, von denen jede mit einem Hall-Effekt-Sensor ausgestattet ist. Diese Sensoren erfassen die Position eines Magneten, der im Puck integriert ist. Sobald der Puck über einen bestimmten Spot bewegt wird, schaltet sich dieser aus, und ein neuer Spot leuchtet auf.

Ergebnisse	Eine spielfähige Oberfläche, die mit vier Messplatinen und einem speziellen Spielmodus gezielte Trainingseinheiten zur Verbesserung des Stick-Handlings ermöglicht, wurde entwickelt. Die Steuerplatine übernimmt sowohl die Ansteuerung der einzelnen Spots als auch die Auswertung der Messdaten. Der aktuelle Score kann auf einem über Wi-Fi gehosteten Webserver angezeigt werden.
Typische Grafik	
Teilnahme an Wettbewerben; Auszeichnungen	Jugend Innovativ
Möglichkeiten der Einsichtnahme in die Arbeit	Die Diplomarbeit ist in Digitalform gespeichert. Diese ist am SharePoint-Server der HTBLuVA Salzburg abgelegt.

Diploma Thesis Documentation

Author(s)	Iason Vasiliadis (Project Leader) Fabian Gassler
Form / Academic year	5AHEL, 2024/2025
Topic	PuckVision - Development of an ice hockey training system
Co-operation partners	-
Assignment of Tasks	Ice hockey training is complex and expensive - PuckVision offers a comfortable alternative. The training module allows you to improve stick handling off the ice. The possibility of quick assembly and space-saving storage make it even more attractive. Different modes constantly offer new challenges.
Realisation	Beneath the playing surface, which consists of a plexiglass sheet, there are multiple sensor boards, each equipped with a Hall-effect sensor. These sensors detect the position of a magnet embedded in the puck. When the puck moves over a specific spot, it turns off, and a new spot lights up.

Results	A playable surface has been developed, featuring four sensor boards and a dedicated game mode that enables targeted training sessions to improve stick handling. The control board manages both the activation of individual spots and the evaluation of measurement data. The current score can be displayed on a web server hosted via Wi-Fi.
Illustrative Graph	
Participation in Competitions; Awards	Jugend Innovativ
Accessibility of Diploma Thesis	The thesis is stored in digital form and is available on the SharePoint server of HTBLuVA Salzburg.

Vorwort

Wir leben in einer Welt, die zunehmend durch Fortschritt und Innovation geprägt ist. So kommt es, dass die Technik zunehmend auch auf Themen wie Sport übergreift, in denen digitale Lösungen und moderne Technologien eine immer zentralere Rolle spielen.
Als unser jetziger Betreuer, Prof. Dr. Tech. Bernhard Prommegger, gegen Ende des vierten Jahrgangs ein Projekt vorschlug, bei dem Sport und Technik verschmelzen, schien dies die perfekte Aufgabe für uns zu sein. Mit unserer Diplomarbeit „PuckVision“haben wir es uns zum Ziel gesetzt, einen weiteren bedeutenden Schritt in diese Richtung zu setzen. Mit großer Motivation und Interesse haben wir unser Hauptaugenmerk darauf gerichtet, ein schnelles und komfortables Training für jedermann von zu Hause aus zu gewährleisten.

Salzburg, am 25. März 2025
Iason Vasiliadis & Fabian Gassler

Abstract-Deutsch

Die Diplomarbeit befasst sich mit der Entwicklung des Eishockey-Trainingssystems PuckVision. Als Spielfläche dient eine 80 cm × 80 cm große Kunststoffplatte, unter der sich einige „Spots "befinden. Das Ziel ist das Stick-Handling und die Reaktionsgeschwindigkeit der Spieler zu optimieren, indem der Puck möglichst schnell über aufleuchtende Spots bewegt wird. Jeder Spot besteht aus einer RBG Leuchtdiode und einem analogen Halleffektsensor, der das Magnetfeld des Pucks, der einen Magneten enthält, erfasst und überprüft, ob dieser über die markierte Position geführt wurde. Durch eine gezielte Lichtsteuerung wird die Reihenfolge der Spots dynamisch vorgegeben, wodurch variierende Trainingsszenarien ermöglicht werden. Die erfassten Sensordaten werden in Echtzeit über das serielle Kommunikationsprotokoll I²C (Inter-Integrated Circuit) an eine zentrale Steuereinheit weitergeleitet. Diese wird durch einen ESP32-Mikrocontroller realisiert, der sämtliche Sensor- und LED-Signale verarbeitet, synchronisiert und die Spiellogik steuert. Zusätzlich werden die Trainingsdaten an eine begleitende Webanwendung übermittelt. Diese enthält eine Datenanzeige für den Punktestand sowie eine Option zum Zurücksetzen des Spiels.
Durch die Kombination aus Sensortechnik, Datenverarbeitung in Echtzeit und einer interaktiven Benutzeroberfläche stellt „PuckVision“ein effizientes Trainingssystem dar, das ein optimiertes Training ermöglicht.

Abstract-Englisch

The thesis focuses on the development of the ice hockey training system PuckVision. The playing surface consists of an 80 cm × 80 cm plastic plate, under which several „spots “are located. The goal is to optimize stick handling and the players' reaction speed by moving the puck as quickly as possible over illuminated spots.
Each spot consists of an RGB light-emitting diode and an analog Hall effect sensor, which detects the magnetic field of the puck, which contains a magnet, and verifies whether it has been guided over the marked position. Through targeted light control, the sequence of the spots is dynamically determined, enabling various training scenarios.
The collected sensor data is transmitted in real time via the serial communication protocol I²C (Inter-Integrated Circuit) to a central control unit. This is implemented using an ESP32 microcontroller, which processes and synchronizes all sensor and LED signals while managing the game logic.
Additionally, the training data is sent to an accompanying web application, which provides a data display for the score as well as an option to reset the game.
By combining sensor technology, real-time data processing, and an interactive user interface, ”PuckVision”provides an efficient training system that enables optimized practice sessions.

Danksagung

An dieser Stelle möchten wir uns herzlich bei all jenen bedanken, die uns im Laufe der Entwicklung von PuckVision zur Seite gestanden sind und uns immer wieder motiviert haben. Ein besonderer Dank geht an unseren Projektbetreuer Prof. Dr. techn. Bernhard Prommegger, dessen zahlreiche Ratschläge sowie seine ausführliche Planung maßgeblich zum Erfolg des Projekts beigetragen hat. Weiters möchten wir uns bei Ing. Dipl.-Päd. FL Helmut Strasser und Ing. Dipl.-Päd. FL Norbert Wen bedanken, die uns mit ihrer Expertise sehr viel weitergeholfen haben. Zu guter Letzt bedanken wir uns herzlich bei unseren Freunden und Familien, die uns während unserer gesamten Schullaufbahn emotional zur Seite standen.

Inhaltsverzeichnis

1 Projektüberblick **1**

1.1 Zielsetzung 1

1.2 Umsetzung 2

2 Hardware **4**

2.1 Systemübersicht 4

2.2 Hall Effekt Sensoren 5

2.3 Grundlagen 5

2.4 Hardware des Sensors 6

2.5 WS2812B 7

2.6 Steckverbinder 10

2.7 Wahl des Mikrocontrollers 11

3 Leiterplattenstruktur **13**

3.1 Prototyp 13

3.2 Schieberegister 15

3.3 SML0805 16

3.4 Fazit 16

3.5 Platinendesign 17

4 Übertragungsprotokolle **22**

4.1 I²C Protokoll 22

4.2 Neopixel Protokoll 23

4.3 HTTP 24

5 Firmware **25**

5.1 I²C Kommunikation 26

5.2 Webserver 31

6 Software **33**

6.1 Ablauf der Software 33

6.2 Frontend 34

6.3 Benutzeroberfläche 35
6.4 Backend 36

7 Fazit und Ausblick **41**
7.1 Probleme und Lösungen 41
7.2 Ausblick 41

Quellenverzeichnis **45**

Begleitprotokolle **47**

1 Projektüberblick

Im Rahmen des Projektes besteht die Aufgabe, ein Eishockeytrainingssystem zu entwickeln, das als Alternative zu herkömmlichem Eishockey-Präzisions- und Reaktionstraining dient. Eine zentrale Herausforderung dabei ist die Gestaltung einer Spieloberfläche, die einerseits den Puck präzise bewegen kann und andererseits sicherstellt, dass die aufleuchtenden Lichter für den Benutzer gut erkennbar sind. Es muss gewährleistet werden, dass die Benutzer intuitiv mit den Lichtern und dem Puck interagieren können, ohne dass falsche oder zu frühe Reaktionen auftreten.
Ein weiteres Problem besteht in der präzisen Platzierung der Sensoren und Leuchtdioden unter der Spielfläche, sodass verschiedene „Spots“ gezielt beleuchtet werden können, wenn der Puck in deren Nähe ist. Die Herausforderung liegt darin, dass der Sensor nur dann reagieren soll, wenn sich der Puck im Radius von zwei bis drei Zentimetern um den Sensor befindet, ohne eine zu frühe Auslösung. Diese präzise Abstimmung zwischen Sensor und Puck ist erforderlich, um eine realistische Spiellogik zu gewährleisten. Zudem ist eine schnelle und effiziente Datenverarbeitung notwendig, die in Echtzeit erfolgt, um einen möglichst realistischen Spielfluss zu ermöglichen. Die Echtzeit-Datenverarbeitung stellt die Herausforderung, Verzögerungen und Latenzen zu minimieren, um die Reaktionsfähigkeit des Systems zu sichern. Außerdem soll eine benutzerfreundliche, gut sichtbare Anzeige entwickelt werden, die in Echtzeit relevante Daten wie Spielergebnisse oder Trainingsfortschritte liefert, ohne den Spielfluss zu beeinträchtigen.

1.1 Zielsetzung

Ziel dieses Projekts ist der Nachbau eines Eishockey-Trainingssystems nach dem Vorbild des Modells der Firma GameChanger [1]. Das System soll eine platzsparende und kostengünstige Lösung bieten, die gleichzeitig einen flüssigen, möglichst realitätsnahen Spielfluss gewährleistet. Dabei liegt der Fokus auf einer benutzerfreundlichen Anwendung und einer praxisgerechten Umsetzung für effektives Training.

[1] https://gamechangerhockey.com

1.2 Umsetzung

Die Spielfläche, siehe Abbildung 1.1, wurde mit einer 80x80 cm Plexiglasscheibe realisiert. Das Material wurde so gewählt, dass einerseits sich der Puck möglichst flüssig über die Oberfläche manövrieren lässt und andererseits die leuchtenden Spots aus Spielersicht gut erkennbar sind. Unter der Spielfläche befindet sich eine Schaumstoffschicht, durch Seitenwände begrenzt, in die entsprechenden Plätze für die Platinen nach der Systemstruktur, siehe 1.2.2, eingeschnitten sind. Insgesamt wurden vier Spots verbaut.

1.2.1 Spielablauf

Zu Spielbeginn leuchtet ein Spot auf. Sobald der Puck über diesen bewegt wird, wechselt das Licht per Zufallsgenerator an einen anderen der vier Plätze. Der Spieler muss nun diesen Vorgang in einer vorgegebenen Zeit so oft wie möglich wiederholen. Die Anzeige des Spielstandes, siehe 6.2, kann auf einer Webseite eingesehen werden.

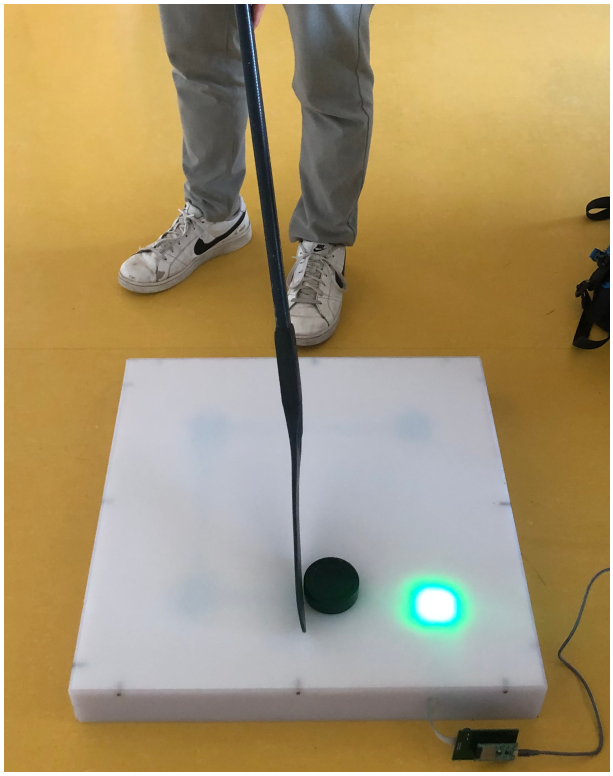


Abbildung 1.1: Spieloberflaeche

1.2.2 Systemstruktur

Das System, siehe Abb. 1.2, besteht auf der Hardwareseite aus einer Steuerplatine und mehreren Messplatinen. Auf der Steuerplatine sitzt der Mikrocontroller ESP32, welcher als zentrales Element den Ablauf und die Verbindung steuert. Dieser wird über seinen Mikro-USB-Port mit Strom versorgt. An die Steuerplatine sind die Messplatinen (Spots) in Reihe angeschlossen.
Softwaretechnisch wird das System in drei Teilbereiche gegliedert. Einerseits der I²C-Bus (siehe Kapitel 4.1) mit dem ESP32 als Master und den Sensoren als Slaves, andererseits die RBG LEDs,die ebenfalls in Serie geschalten und bilden eine Art „LED-Kette“, wie genauer in 2.5 ausgeführt. Der letzte Teil besteht aus der Webanwendung, wobei der ESP selbst als Webserver fungiert und eine Webseite hostet, wie in Kapitel 5.2 näher erläutert.

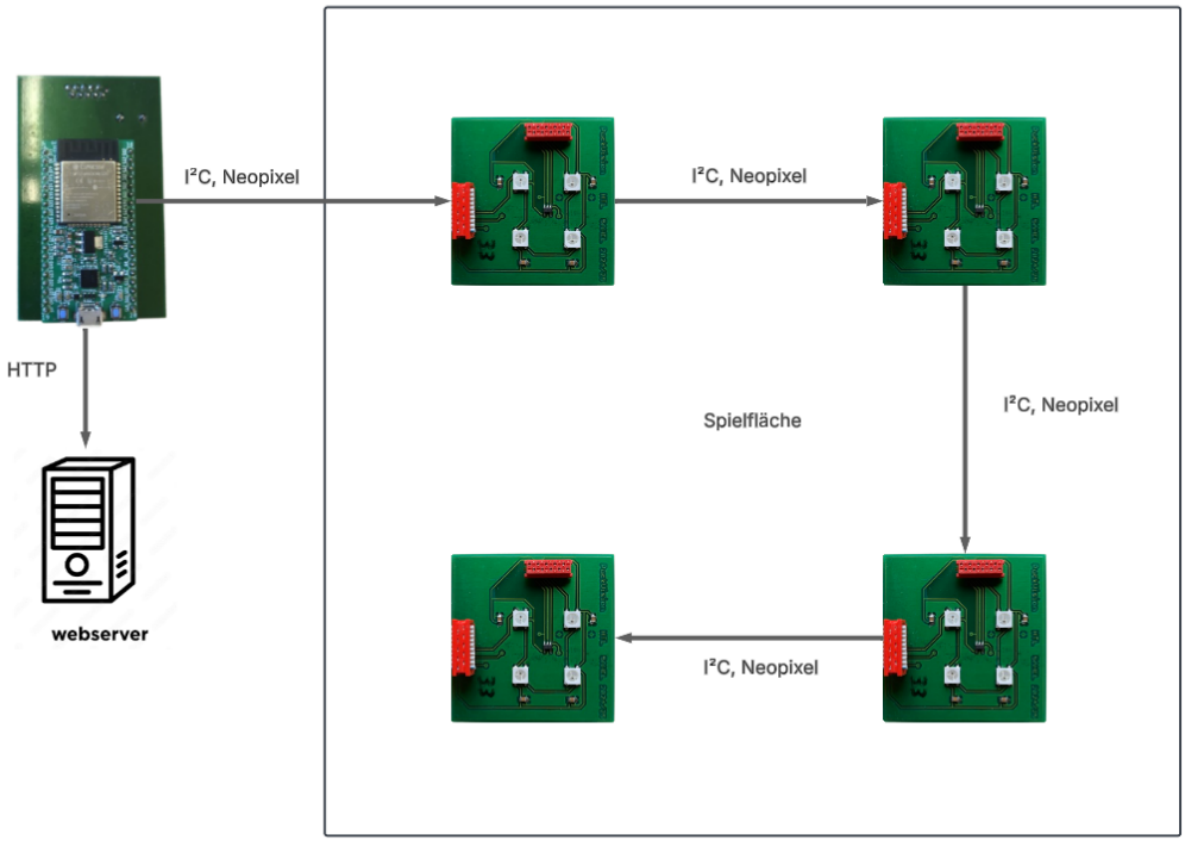


Abbildung 1.2: Blockschaltbild Programmstruktur

2 Hardware

Die Hardware von PuckVision gewährleistet die präzise Erfassung der Puck-Position auf dem Spielfeld sowie die Fertigung der Spieloberfläche selbst. Diese Daten werden durch eine zentrale Steuerplatine verarbeitet, die eine Vielzahl von Messplatinen ansteuert und so eine zuverlässige Datenerfassung ermöglicht.

2.1 Systemübersicht

PuckVision kann in zwei zentrale Hardwarekomponenten aufgeteilt werden:
1) Die Steuerplatine enthält einen Mikrocontroller, der für die Ansteuerung der Messplatine und die Aufnahme der Werte verantwortlich ist.
2) Die Messplatinen erfassen die Daten der Spieloberfläche und senden sie zur weiteren Verarbeitung an die Steuerplatine.
Das in Abbildung 2.1 dargestellte Blockschaltbild visualisiert die Zusammenhänge der einzelnen Komponente. Wie oben bereits erwähnt sind die zwei zentralen Bausteine die Steuerplatine und die Messplatine. Die Versorgung erfolgt über den ESP32 (auf der Steuerplatine) und ist mit VCC und GND gekennzeichnet. Der ESP32 ist außerdem für die Datenübertragung der LEDs und für die Auslesung der Messwerte des Hall Effekt Sensors. Ein Webserver wird via Wi-Fi gehostet und zeigt den aktuellen Spielstand an.

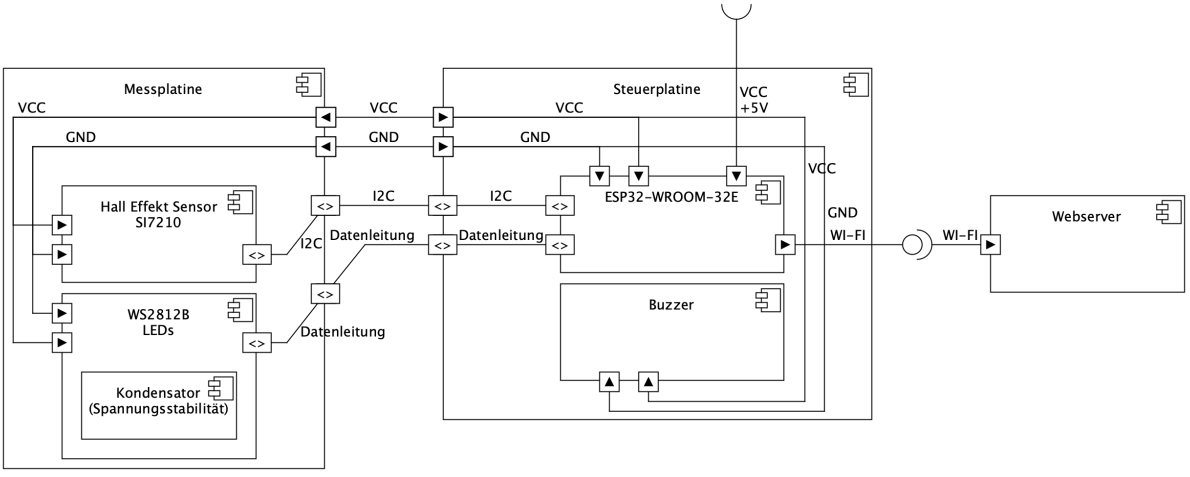


Abbildung 2.1: Blockdiagramm

Die vorliegende Arbeit beschreibt alle relevante Komponenten für ein besseres Verständnis. In Folge wird ein Prototyp beschrieben und anschließend die Mess- & Steuerplatine. Der Hall Effekt Sensor wurde unter Abschnitt 2.2 beschrieben. Die WS2812B LEDs sind unter 2.5, die Steckverbinder unter 2.6 und der ESP32 unter 2.7.2 zu finden.

2.2 Hall Effekt Sensoren

Die SI7210 I²C Hall Effekt Sensoren von Silicon Labs wurden verwendet um die Position des Pucks in den für uns relevanten bereichen auf der Spieloberfläche (über den Spots) zu ermitteln. Die Abbildung 2.2 veranschaulicht den Aufbau des Hallsensors inform eines Blockschaltbildes. Der Sensor bietet eine sehr hohe Empfindlichkeit, einen 13-Bit-ADC für hohe Genauigkeit und eine digitale Temperaturkompensation. Über die I²C-Schnittstelle (SCL, SDA) lassen sich Schaltpunkte konfigurieren und die 13-Bit-Magnetfeldstärke auslesen. Der Sensor ermöglicht präzise Messungen auch bei großen Luftspalten und kleinen Magneten. [1]

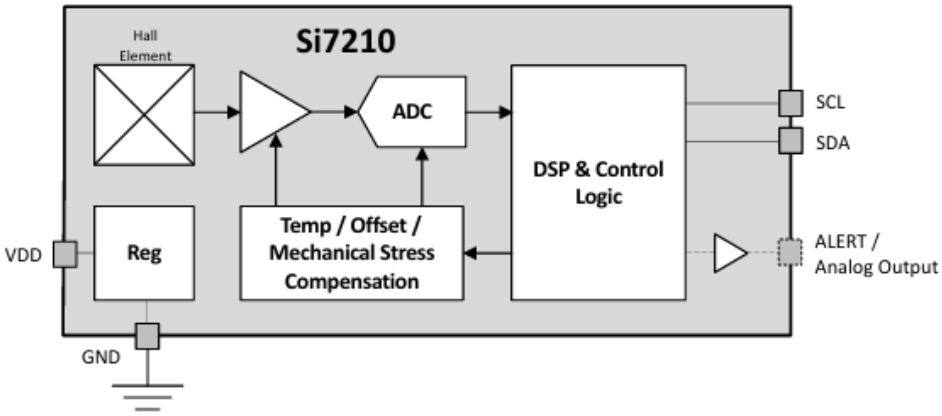


Abbildung 2.2: Si7210 Hall Effekt Sensor [1]

2.3 Grundlagen

Der Halleffekt beschreibt die Entstehung einer elektrischen Spannung quer zum Stromfluss in einem Leiter, wenn dieser einem Magnetfeld ausgesetzt ist. Diese Spannung kann genutzt werden, um die Magnetfeldstärke zu messen. Der Mikrocontroller erhält

innerhalb kürzester Zeit die Werte der Magnetstärke und ermittelt somit die Position des Pucks.

2.3.1 Aufbau

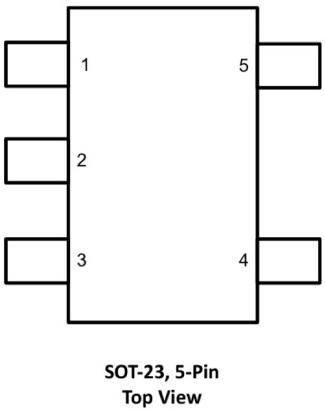
Der Si7210 vereint ein Hallelement mit einer niederrauschenden Verstärkerschaltung und einem 13-Bit-Analog-Digital-Wandler. Die wichtigsten technischen Merkmale sind:

- Hohe Empfindlichkeit (bis zu ±20 mT oder ±200 mT einstellbar)
- Digitale Signalverarbeitung zur Temperatur- und Offsetkompensation
- I²C-Schnittstelle zur Kommunikation mit Mikrocontrollern
- Niedriger Stromverbrauch (50 nA im Schlafmodus)

2.4 Hardware des Sensors

Nachfolgend werden die wichtigsten Hardwareeigenschaften des Hall Effekt Sensors erklärt.
Bei der Inbetriebnahme müssen folgende elektrischen Eigenschaften beachtet werden:
1) Eine Versorgungsspannung zwischen 1,7V - 5,5V muss eingehalten werden.
2) Die Betriebstemperatur sollte -40 bis +125 °C betragen.
3) Das Ausgangssignal kann Digital oder Analog ausgelesen werden.

Der Sensor ist in den Gehäusen SOT-23 und DFN verfügbar. Wir benutzen die SOT-23 Bauform welcher in der Abbildung 2.3 dargestellt ist..



Pin Name	Pin Number	Description
SDA	1	I²C data
GND	2	Ground
SCL	3	I²C clock
V_{DD}	4	Power +1.7 to +5.5 V
ALERT/VOUT	5	Analog or digital ouput

Abbildung 2.3: Pinbelegung Top View [1, S. 21]

Ein typisches Schaltbild zur Anbindung an einen Mikrocontroller umfasst Pull-up-Widerstände an SDA und SCL sowie eine stabile Spannungsversorgung. Darauf wird in weiterer Folge noch eingegangen.

2.4.1 Hardwareadressen

Die Hardwareadressen des Si7210 Halleffektsensors beziehen sich auf die I²C-Adressen, die für die Kommunikation mit dem ESP32 Mikrocontroller verwendet werden. Der Si7210 verwendet eine 7-Bit I²C-Adresse. Die genaue Adresse hängt vom spezifischen Modell des Sensors ab. Die Bauteilnummer des Hall Effekt Sensors kann wiefolgt aussehen: **SI7210-B-00-IV**. SI7210 ist die Grundbezeichnung. Der Buchstabe „B "beschreibt die Magnetempfindlichkeit, „00 "die Art des Gehäuses und „IV "den Temperaturbereich des Sensors. [1] Die Adressen werden hardwareseitig vorgegeben und können nicht per Software geändert werden. Laut dem Datenblatt sind die möglichen festen I²C-Adressen: 0x30, 0x31, 0x32 und 0x33.

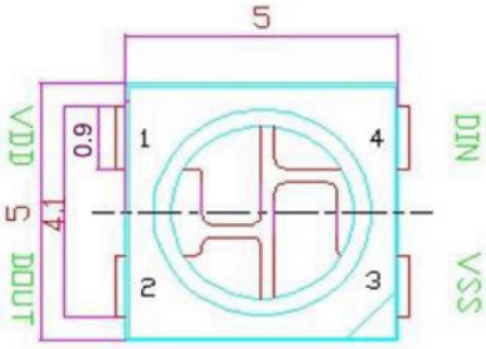
2.4.2 I²C-Kommunikation mit der Hardwareadresse

Die I²C-Kommunikation mit den Hardwareadressen erfolgt nach dem Master-Slave-Prinzip. Eine Auswahl an Hardwareadressen ermöglicht dem Master (Microkontroller) eine gezielte Ansteuerung mehrerer Slaves (Sensoren), welche an der selben I²C-Leitung angeschlossen sind. Durch die Verwendung unterschiedlicher Hardwareadressen können mehrere Sensoren einfach in Reihe betrieben werden, ohne dass zusätzliche Hardware erforderlich ist. Falls eine Erweiterung erwünscht ist, wäre ein I²C-Multiplexer nötig, um mehrere separate I²C-Leitungen zu erzeugen und die Kommunikation mit baugleichen Sensoren zu ermöglichen.

2.5 WS2812B

Die WS2812B ist eine intelligent gesteuerte LED-Lichtquelle, bei der die Steuerschaltung und der RGB-Chip in einem Gehäuse von 5050-Komponenten integriert sind. Eine Gehäuseform von 5050 bedeutet, dass das Bauteil quadratisch aufgebaut ist, mit einer Seitenlänge von 5mm, was in der Abbildung 2.4 ersichtlich ist. Die WS2812B LED kann alle Farben (RGB), in unterschiedlicher Intensität darstellen. Sie enthält

intern ein intelligentes digitales Port-Datenlatch und eine Signalverstärkungsschaltung. Zudem verfügt sie über einen präzisen internen Oszillator und einen 12V-Spannungs-programmierbaren Konstantstromregelungsteil, wodurch eine hohe Farbkonsistenz der Pixelpunkte effektiv gewährleistet wird.



NO.	Symbol	Function description
1	VDD	Power supply LED
2	DOUT	Control data signal output
3	VSS	Ground
4	DIN	Control data signal input

Abbildung 2.4: WS2812B Gehäuse mit Pinbelegung [2, S. 2]

Das Datenübertragungsprotokoll verwendet einen einzelnen NZR- Kommunikations-modus, welcher der Software zugewiesen wird und daher unter dem Abschnitt 4.2 beschrieben wurde. Die LED zeichnet sich durch eine niedrige Betriebsspannung, Umweltfreundlichkeit und Energieeffizienz, hohe Helligkeit, einen großen Streuwinkel, gute Konsistenz, geringen Stromverbrauch und eine lange Lebensdauer aus. Der in der LED integrierte Steuerchip macht die Schaltung einfacher, das Volumen kleiner und die Installation bequemer [2, S. 1].

2.5.1 Pinbelegung und technische Daten

Die WS2812B LEDs besitzen vier Pins, welche in der Abbildung 2.4 nummeriert sind. Diese sind in der nebenstehenden Tabelle benannt und beschrieben. Die in der Abbildung 2.6 aufgelisteten technischen Daten sind vom Hersteller vorgeschrieben. „Power supply voltage“ist die vorgeschriebene Betriebsspannung und „Operation junction temperature “ ist die Betriebsübergangstemperatur. Diese Werte sind unbedingt einzuhalten.

2.5.2 Schaltungsaufbau

In der Abbildung 2.5 ist der Schaltungsaufbau der WS2812B LEDs zu sehen. Die LEDs können in einer Kette verbunden werden, wobei der DOUT-Pin einer LED mit dem

DIN-Pin der nächsten LED verbunden wird. Dies ermöglicht die Steuerung einer großen Anzahl von LEDs mit nur einem Datenpin des Mikrocontrollers.

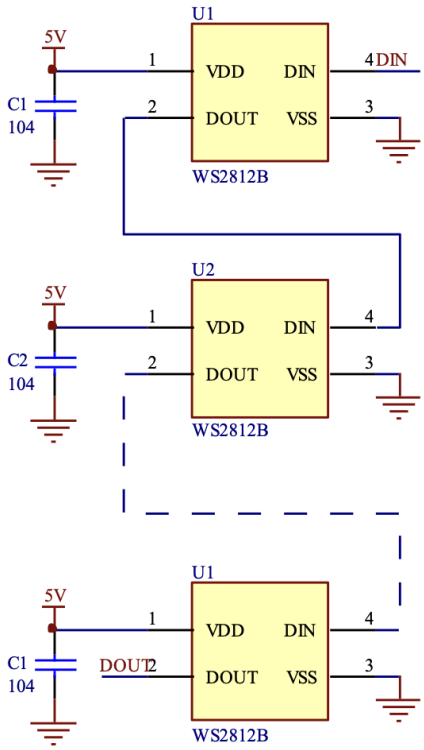


Abbildung 2.5: WS2812B Schaltung [2, S. 5]

Prameter	Symbol	Ratings	Unit
Power supply voltage	V_{DD}	+3.5~+5.3	V
Input voltage	V_I	-0.5～VDD+0.5	V
Operation junction temperature	Topt	-25～+80	℃
Storage temperature range	Tstg	-40~+105	℃

Abbildung 2.6: WS2812B Absolute Höchstwerte[2, S. 2]

2.6 Steckverbinder

Das Bauteil mit der Bezeichnung WR-MM-690367181072 ist ein Steckverbinder in der Board zu Board Bauform und besteht aus einer weiblichen und männlichen Komponente. Dieser Connector ist von der Montageart ein THT-Bauteil (Through Hole Technology) und besitz keine Polarisation [3]. In Abbildung 2.7 ist sowohl die weibliche (links) als auch die männliche (rechts) Komponente abgebildet.

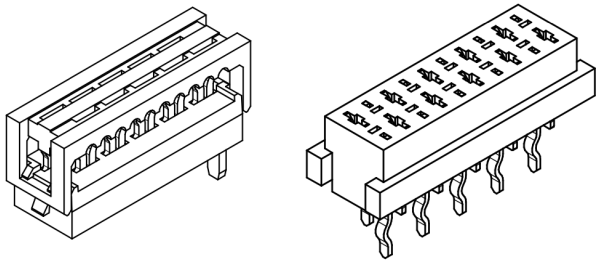


Abbildung 2.7: female/male Steckverbinder [3]

Der WR-MM MiniModule Connector ermöglicht platzarme Board-Verbindungen und optimiert den Sensor-Magnet-Abstand durch seine Bauform. Die horizontale Steckverbindung verhindert unnötige Höhen.

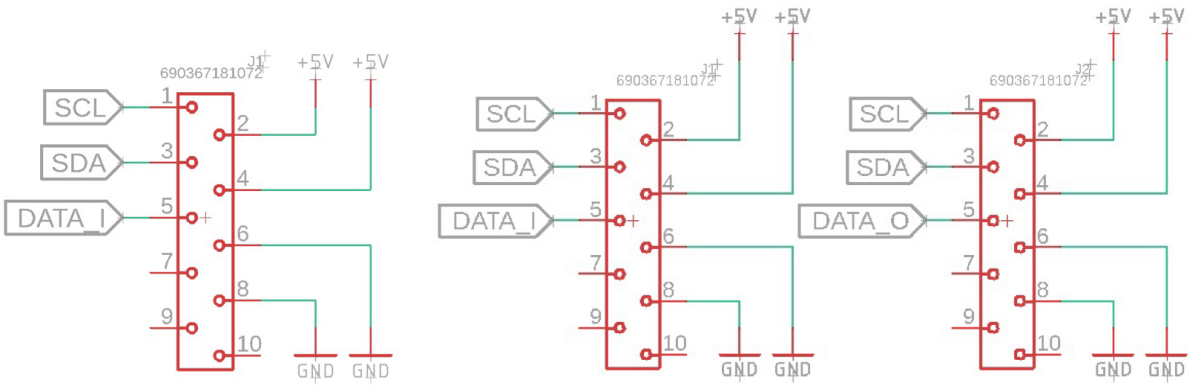


Abbildung 2.8: Symbol Eagle Steuer- & Messplatine

Die in Abbildung 2.8 dargestellte Anordnung verteilt sich auf die Steuer- und die Messplatine. Das linke Symbol befindet sich auf der Steuerplatine, wie in Abbildung 3.5 zu sehen ist. Das mittlere und rechte Symbol sind auf der Messplatine verbaut, wie in Abbildung 3.7 dargestellt. Zur besseren Verständlichkeit dieser Anordnung sollte erneut auf den Schaltungsaufbau der WS2812B-LEDs verwiesen werden (siehe Abbildung 2.5). Das rechte Eagle Symbol unterscheidet sich von den anderen am Pin 5. Dabei handelt es sich um die Datenleitung DATA_I / DATA_O. Diese Leitung

durchläuft alle verbauten LEDs und ermöglicht ihre Programmierung. Die Datenleitungen SDA&SCL dienen der I²C-Kommunikation, welche die Werteaufnahme der Halleffektsensoren regelt. Die I²C-Kommunikation ist genauer in Abschnitt 4.1 und der Halleffektsensor in Abschnitt 2.2 beschrieben. Bei der Pinbelegung wurde darauf geachtet, dass Datenleitungen jeweils durch eine GRUND- oder VCC-Leitung getrennt sind. Dies dient der Schirmung der Datenleitungen, um störende Einflüsse zu minimieren.

2.7 Wahl des Mikrocontrollers



ESP32-WROOM-32E

Abbildung 2.9: ESP32-WROOM-32E [4]

2.7.1 Anforderungen

Nach einer detaillierten Analyse der Problemstellung wurden folgende Anforderungen an den Mikrocontroller festgelegt:
Da alle erforderlichen Kriterien im uns von der Schule zur Verfügung gestellten Mikrocontroller „ESP32-WROOM-32E-N4“ erfüllt werden, fiel die Wahl auf ihn. Dieser wird im Abschnitt 2.7.2 genauer beschrieben.

- **I²C-Kommunikation:** Der Mikrocontroller muss I²C unterstützen, um die Kommunikation mit den einzelnen Platinen zu ermöglichen. Freie I²C-fähige Pins sind erforderlich, um zukünftige Erweiterungen des Systems zu unterstützen.
- **32-bit Architektur:** Ein 32-Bit Mikrocontroller wurde gewählt, um eine schnelle und effiziente Verarbeitung der Messdaten zu gewährleisten, was für die Echtzeitanforderungen des Projekts entscheidend ist.

- **Integriertes Wi-Fi-Modul:** Der Mikrocontroller muss über ein integriertes Wi-Fi-Modul verfügen, um einen Webserver zu hosten, der den Spielstand in Echtzeit darstellt.
- **Energiesparmodi:** Der Mikrocontroller sollte über eine Vielzahl von Energiesparmodi (Deep Sleep (größer 10µA), Light Sleep) verfügen, um ihn für batteriebetriebene Anwendungen geeignet zu machen.

2.7.2 Mikrocontroller ESP32

Der gewählte Mikrocontroller „ESP32-WROOM-32E-N4“ erfüllt alle in Abschnitt 2.7.1 festgelegten Kriterien. Wichtige technische Daten des Chips [4, S. 2]:

- **Empfohlene Betriebsbedingungen:** Spannung: 3.0V - 3.6V (Typ = 3.3V)
- **Kerne:** Dual-Core
- **Wi-Fi Betriebskanal:** Frequenzbereich: 2412 - 2484 MHz
- **Wi-Fi Wireless Standard:** IEEE 802.11b/g/n
- **Bluetooth:** Version: V4.2 BR/EDR und Bluetooth LE
- **Flash-Speicher:** Größe: 4MB (Quad SPI)
- **Taktrate:** Bis zu 240 MHz

2.7.3 Wahl der Pinbelegung

Der ESP32 verfügt über 34 GPIO-Pins, denen durch Programmierung der entsprechenden Register verschiedene Funktionen zugewiesen werden können. Es besteht die Möglichkeit, einen Pin als digital-only, analog-enabled oder capacitive-touch-enabled zu betreiben [4, S. 17]. Alle verbauten Pins haben verschiedene Versorgungsmöglichkeiten. Wesentlich beschränkt man sich jedoch beim ESP32 auf 3.0V bis 3.6V. Weitere Details können im Datenblatt unter der Überschrift „Power Domain“ entnommen werden [4]. Die GPIO-Pins 34-39 sind sogenannte input-only Pins. Diese sind für unsere Zwecke unbrauchbar, daher wurden sie auch frei gelassen [4, S. 61]. Für die I²C-Kommunikation wurden die GPIO-Pins 21 (SDA) und 22 (SCL) gewählt, da sie über die notwendigen Pull-up-Widerstände verfügen und eine stabile Kommunikation gewährleisten.

3 Leiterplattenstruktur

Es wurden eine vielzahl an Platinen gefertigt. Ein Prototyp, eine Steuerplatine und acht Messplatinen. Wie bereits angeschnitten, dient die Steuerplatine der Steuerung und Auswertung der aufgenommenen Messdaten. Die Messplatine bestehen aus jeweils einem Halleffektsensor, vier LEDs und zwei Connectoren. Diese drei verschiedenen Platinen sind ein wesentlicher Teil der Hardware und werden in diesem Kapitel genauer beschrieben.

3.1 Prototyp

Um sicherzustellen, dass das technische Konzept funktioniert, wurde ein Prototyp erstellt. Dabei lag der Fokus insbesondere bei der Auswahl der LEDs und der funktionalität des Hall Effekt Sensors. In der Machbarkeitsstudie wurden folgende Fragestellungen bearbeitet:
Ein zentraler Aspekt des Projekts ist die Verbindung mehrerer Platinen, deren Funktionsweise sorgfältig geplant werden muss. Dabei stellt sich die Frage, ob der eingesetzte Halleffektsensor die geforderten Anforderungen hinsichtlich Geschwindigkeit und Genauigkeit erfüllt. Ebenso von Bedeutung ist die Implementierung der I²C-Kommunikation mit dem Sensor, die eine zuverlässige Datenübertragung gewährleisten soll. Darüber hinaus wird geprüft, ob der Einsatz eines Schieberegisters notwendig ist, um die Schaltung effizienter zu gestalten. Schließlich spielt auch die Ansteuerung der WS2812B-LEDs eine wichtige Rolle, da diese in das Gesamtsystem integriert werden müssen. Die Ergebnisse der Studie werden im Abschnitt 3.4 detailiert beschrieben.

3.1.1 Schaltung

Hauptkomponente des Prototypen ist der Mikrocontroller ESP32 2.7.2. Dieser steuert den Halleffektsensor SI7210 an (im Abschnitt 2.2 zu finden). Die LEDs (WS2812B) sind unter Abschnitt 2.5 beschrieben. Weitere LEDs (SML0805) im Abschnitt 3.3 zu finden. Der folgende Abschitt 3.2 behandelt das Schieberegister (74HC595). Der Schaltplan des Prototypen wurde in der Abbildung 3.1 dargestellt.

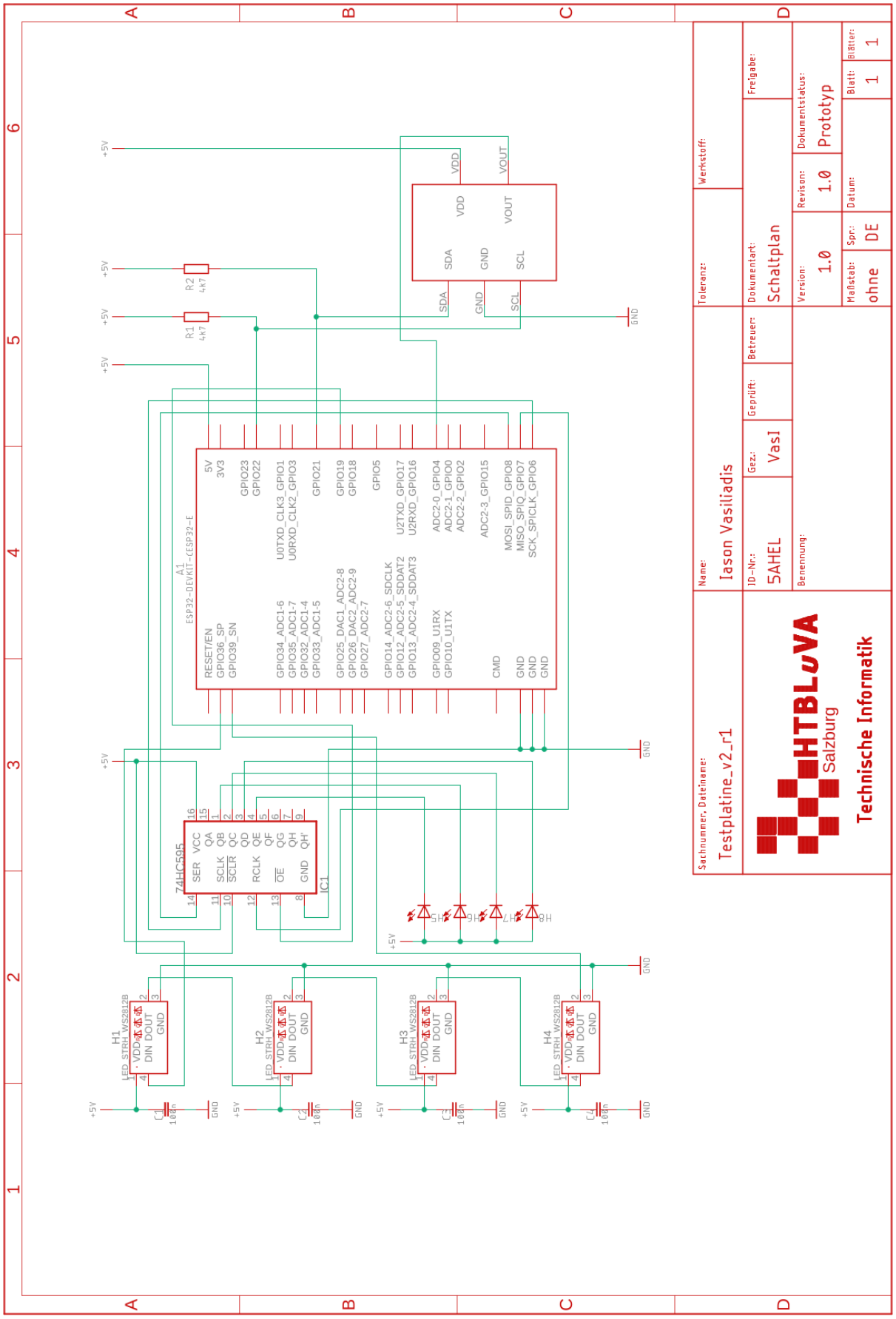


Abbildung 3.1: Schaltung Prototyp Eagle

3.2 Schieberegister

Das Schieberegister 74HC595 in der SMD-Bauform (SurfaceMounted Device) erlaubt es, aus einer geringen Anzahl von GPIO-Pins eines Mikrocontrollers eine Vielzahl von Ausgängen zu generieren[5].

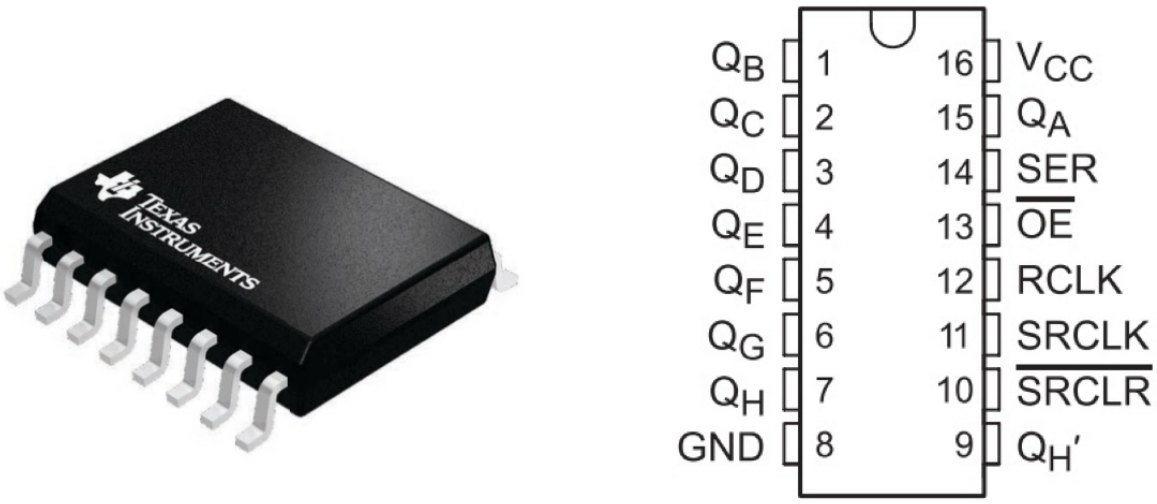


Abbildung 3.2: Schieberegister 74HC595 [5, S. 31]

Der 74HC595 ist ein seriell-zu-parallel Umwandler (Serial-In, Parallel-Out), der es ermöglicht, Daten seriell einzulesen und anschließend parallel auszugeben. Diese Funktionalität ist insbesondere dann vorteilhaft, wenn eine begrenzte Anzahl von GPIO-Pins eines Mikrocontrollers zur Ansteuerung einer größeren Anzahl von Ausgängen genutzt werden soll.

3.2.1 Pinbeschreibung und technische Daten

Die Abbildung 3.2 zeigt die Pinbelegung des Schieberegisters. Die Abbildung **??** gibt genauere Informationen über die Funktionalität der einzelnen Pins. Die Versorgungsspannung (VCC) variiert zwischen 2,0V-6,0V wobei 5V typisch sind. Falls ein Batteriebetrieb in Frage kommen sollte, wäre das 74HC595 Schieberegister geeignet. Der maximale Ausgangsstrom beträgt 35mA und die geeignete Betriebstemperatur ist zwischen -40°C und +125°C vom Hersteller angegeben. Das Schieberegister eignet sich auch aufgrund seiner hohen Schaltgeschwindigkeit [5].

3.3 SML0805

Das Bauteil SML0805 ist eine billige LED in der Surface-mounted-device Bauform, siehe Abbildung 3.3. Die WS2812B LEDs bieten durch die Programmierbarkeit einen ausschlaggebenden Vorteil, daher wurden diese Verbaut. Eine detaillierte Beschreibung ist unter dem Abschnitt 2.5 zu finden.

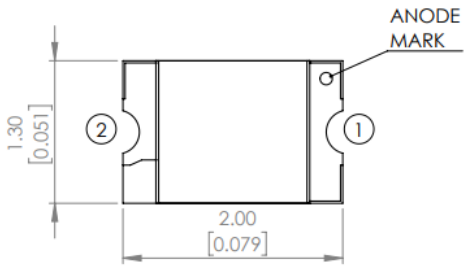


Abbildung 3.3: SML0805 [6]

3.4 Fazit

Der ESP32 erfüllt alle für dieses Projekt relevanten Anforderungen. Die Halleffektsensoren besitzen unterschiedliche Hardwareadressen. Um sie in Serie zu schalten, muss dies unbedingt beachtet werden, da ansonsten ein fehlerhaftes Verhalten zu erwarten ist. Genauere Informationen zu den Hardwareadressen können dem Abschnitt 2.4.1 entnommen werden. Eine I²C-Übertagung besteht aus einer SDA- und SCL-Leitung. Diese brauchen immer jeweils einen Pull-Up-Widerstand. Dies erfolgt inform eines 4k7Ohm Widerstands. Der R1 & R2 sorgen für ein konstantes, rausch und störfreies Signal. Aufgrund der Einzigartigkeit der Sensoren und der internen Adressen der LEDs, ist der Gebrauch eines Schieberegisters unnötig. Die Funktion der WS2812B LEDs ist im Abschnitt 2.5 zu finden. Falls mehr als vier Messplatinen erwünscht sind, wird ein I²C-Multiplexer gebraucht um eine Hardwareadresse öfter benutzbar zu machen. Ein I²C-Multiplexer kann aus einer GPIO-Pin Leitung mehrere I²C-Kanäle zur verfügung stellen. Die WS2812B LEDs sind für unsere Zwecke geeignet. Sie bieten uns eine Vielfalt an Farben und können in Serie geschaltet werden, außerdem sind sie nach belieben programmierbar. Die Hardware der Leds sind unter dem Abschnitt 2.5 zu finden. Das Übertragungsprotokoll der LEDs ist im Abschnitt **??** beschrieben. Jede LED benötigt einen 100nF Kondensator, dieser Stabilisationskondensator verhindert Rauschen und

Störungen des Datensignals. Die LEDs SML0805 unterliegen den WS2812B, darum finden sie keine Verwendung in diesem Projekt. Die Pins GPIO36_SP & GPIO39_SN des ESP32 sind sogenannte Input-Only Pins. Diese können nur Signale Einlesen, jedoch keine Ausgeben. Aus diesem Grund konnten die WS2812B LEDs in Abbildung 3.1 nicht angesteuert werden. Dies wurde Hardwaretechnisch behoben, indem die Leitungen der oben genannten GPIO-Pins getrennt wurden und neu verbunden worden sind.

3.5 Platinendesign

Die Aufteilung in eine Steuerplatine und mehrere Messplatinen wurde vorgenommen, um eine klare funktionale Trennung zu erreichen. Die Steuerplatine übernimmt die zentrale Steuerung und Koordination des Systems, während die Messplatinen speziell für die präzise Erfassung und Verarbeitung von Messdaten optimiert sind. Diese Trennung verbessert nicht nur die Übersichtlichkeit und vereinfacht die Fehlersuche, sondern minimiert auch elektromagnetische Störungen und Übersprechen zwischen den Steuer- und Messsignalen. Dadurch wird die Genauigkeit und Zuverlässigkeit der Messungen erhöht. Zudem ermöglicht die Aufteilung eine optimale Anpassung der Layouts: Die Steuerplatine kann für eine effiziente Signalverteilung ausgelegt werden, während die Messplatinen auf minimale Rauschbeeinflussung und maximale Messgenauigkeit designed wurden.

3.5.1 Aufbau Steuerplatine

Die Steuerplatine übernimmt die zentrale Verarbeitung der Sensordaten und steuert angeschlossene Komponenten an. Die in Abbildung 3.4 gefertigtte Platine wurde durch den in Abbildung 3.5 gefertigt.

- **ESP32:** Zentrales Steuerungselement mit GPIOs, analogen/digitalen Eingängen und SPI/I²C-Schnittstellen. Wichtige Anschlüsse:
  - **SOUND:** Ausgang für Lautsprecher
  - **SCL/SDA:** I²C-Schnittstelle zur Messplatine
  - **DATA_I:** LED-Datenleitung
- **Sound-Sektion:** Besteht aus einem 1µF-Kondensator und einem AL60P-Lautsprecher.

- **I²C-Pull-Up-Widerstände:** Je ein 1kΩ-Widerstand an SDA/SCL für stabile I²C-Kommunikation.
- **Stecker-Sektion:** Verbindet die Messplatinen. Enthält SCL, SDA, DATA, +5V und GND für Kommunikation und Stromversorgung.

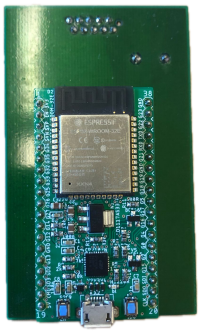


Abbildung 3.4: Steuerplatine

3.5.2 Aufbau Messplatine

Die Messplatine erfasst Sensordaten und leitet sie an die Steuerplatine weiter. Sie ist in verschiedene funktionale Blöcke unterteilt, die jeweils spezifische Aufgaben übernehmen. Die in der Abbildung 3.6 dargestellte Platine entspricht der verwendeten Messplatine.

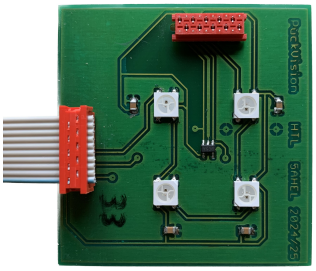


Abbildung 3.6: Messplatine

Die Platine ist in drei Hauptsektionen untergliedert:

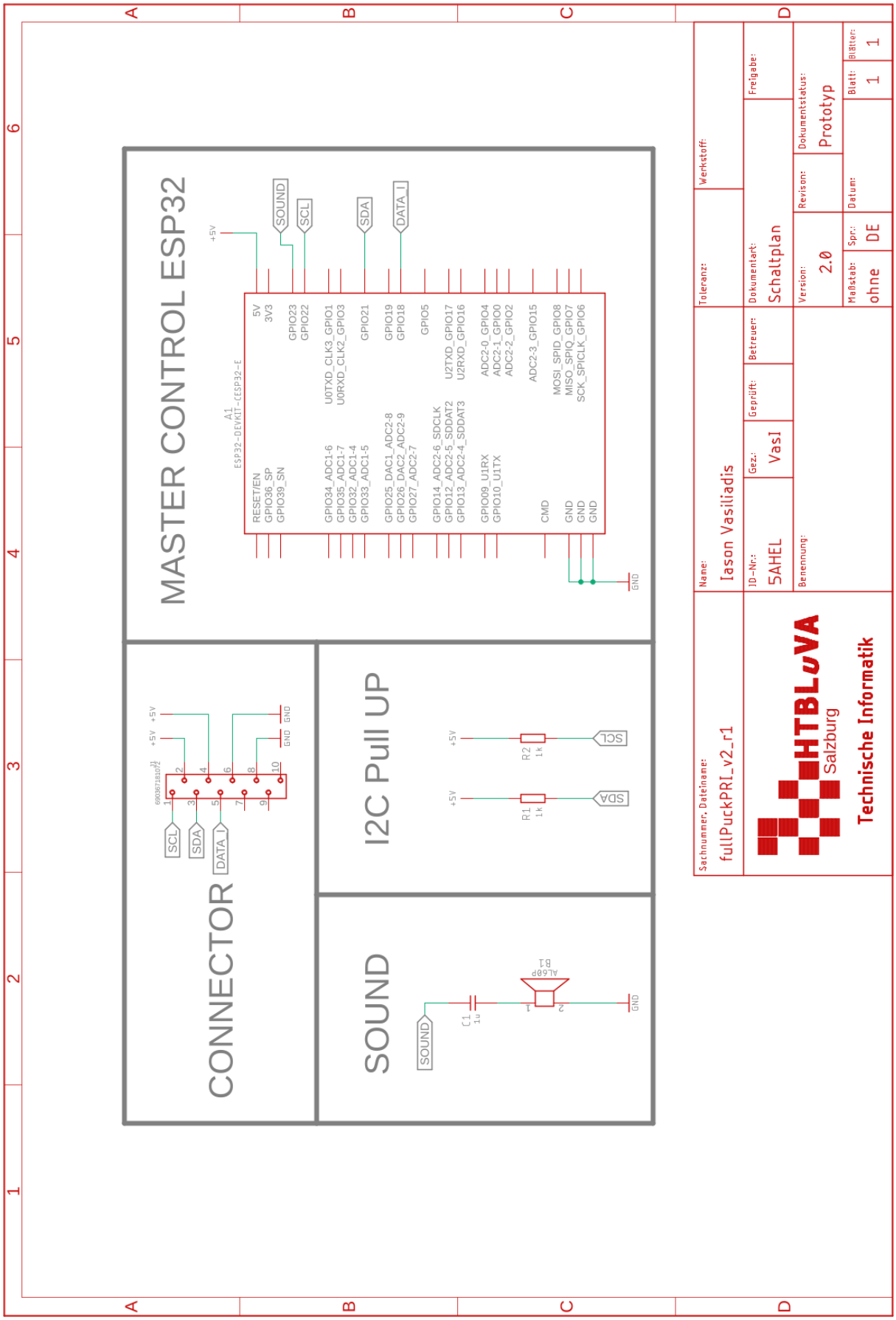


Abbildung 3.5: Steuerplatine

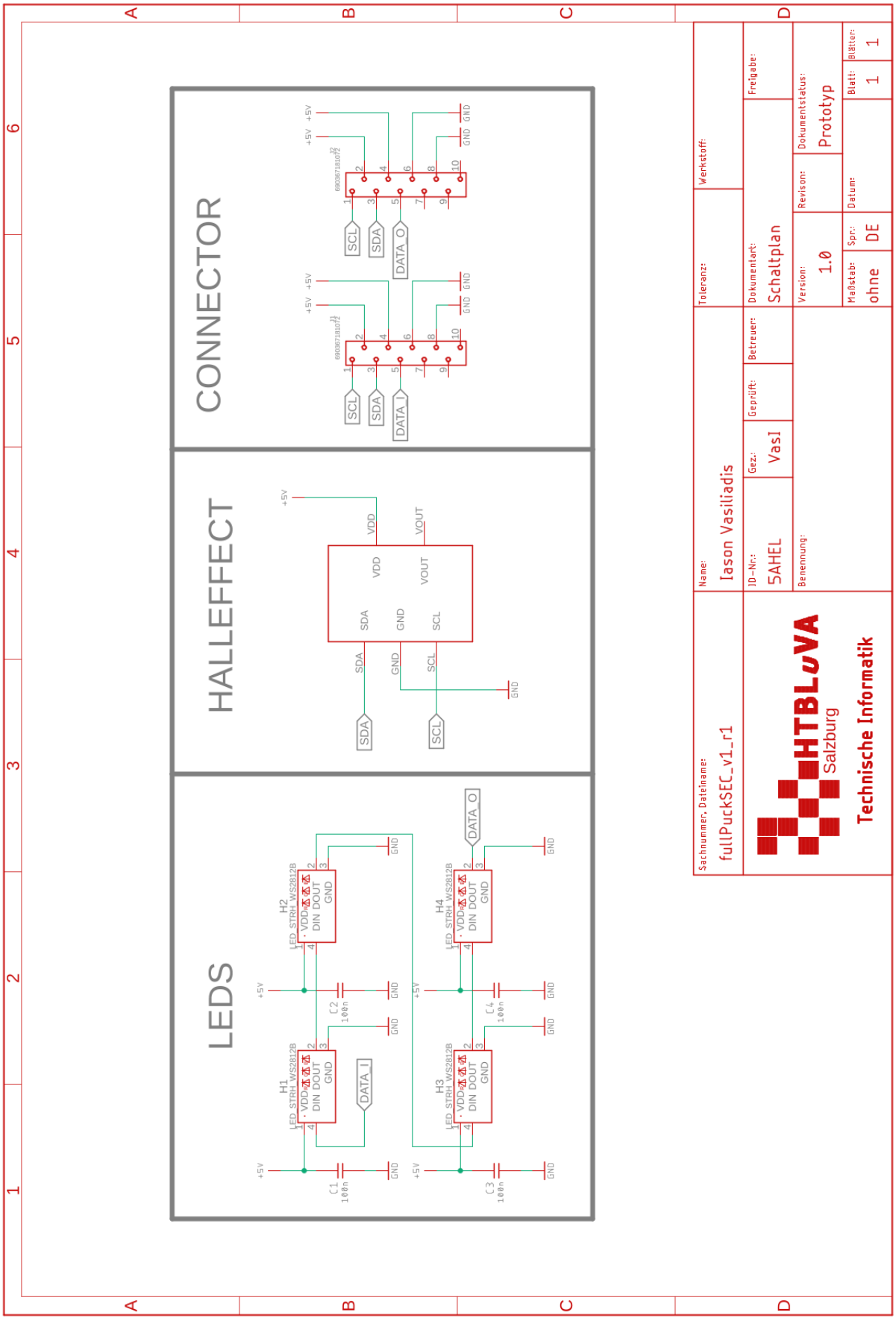


Abbildung 3.7: Messplatine Schaltplan

- **LED-Sektion:** Diese Sektion beinhaltet vier WS2812B- LEDs, die über eine serielle Datenleitung angesteuert werden, siehe Abschnitt 2.5.
- **Hallsensor-Sektion:** Diese Sektion enthält einen Hallsensor, der Magnetfelder misst und ein entsprechendes Ausgangssignal ausgibt, siehe Abschnitt 2.2.
- **Stecker-Sektion:** Diese Sektion enthält zwei Steckerleisten für die Verbindung mit der Steuerplatine oder zur nächsten Messplatine. Die genaue Funktion wurde unter dem Abschnitt 2.6 behandelt.

Auf der folgenden Seite ist in Abbildung 3.7 die zugehörige Schaltung dargestellt. Diese zeigt die Aufteilung der Platine in die drei beschriebenen Sektionen.

4 Übertragungsprotokolle

Übertragungsprotokolle sind standardisierte Verfahren und dienen dem Datenaustausch zwischen Systemen. Hauptsächlich verwendet wurden die Protokolle I²C (4.1), Neopixel (4.2) und HTTP (4.3).

4.1 I²C Protokoll

Der I²C (Inter-Integrated-Circuit-Bus), entwickelt von Philips Semiconductors, ist ein synchroner, serieller Zweidraht Datenbus der die Kommunikation mit Consumer-Elektronik wie Displays, Sensoren und weiteren I²C-kompatiblen Geräten ermöglicht[7].

4.1.1 Aufbau und Kommunikation

Die I²C-Kommunikation nutzt zwei Leitungen, SDA (Serial Data) und SCL (Serial Clock). Der Master (z. B. ESP32) steuert die Kommunikation und gibt den Takt vor, während die Slaves (z. B. Sensoren oder Displays) Daten empfangen oder senden[8]. Die Datenübertragung (siehe Grafik 4.1) erfolgt byteweise mit MSB First, sprich das höchstwertige Bit wird zuerst gesendet. Im Ruhezustand sind SDA und SCL HIGH. Der Master startet die Kommunikation mit einer START-Bedingung, indem er SDA auf LOW zieht, während SCL HIGH bleibt. Anschließend wird die 7-Bit-Hardwareadresse inklusive Read/Write-Bit gesendet, worauf der Slave mit einem Acknowledge-Bit antwortet. Die Übertragung endet mit der STOP-Bedingung, bei der der Master SDA auf HIGH setzt, während SCL weiterhin HIGH ist[8].

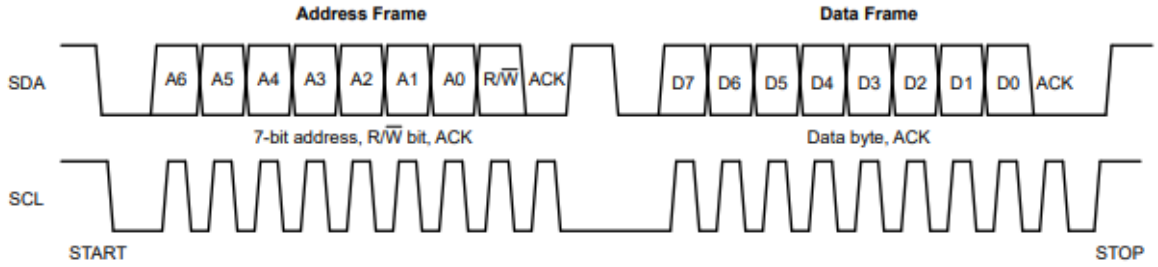


Abbildung 4.1: I²C Datenübertragung.[9]

4.2 Neopixel Protokoll

Das Neopixel Protokoll ist ein, von Adafruit Industries entwickeltes, asynchrones, serielles Kommunikationsprotokoll, das die Ansteuerung von Neopixel LEDs ermöglicht. Dieses Protokoll bietet eine einzelne Adressierung und Steuerung von RBG LEDs in einer LED-Kette.

4.2.1 Aufbau und Kommunikation

KI-Generiert

Das Protokoll verwendet zur Datenübertragung (siehe Abb. 4.2) einzelne Datenleitung (DIN), über die die Farbwerte seriell an die LEDs übertragen werden. Dabei wird das 24-Bit-Datenpaket Bit für Bit gesendet, wobei zwischen den Bits durch unterschiedliche Pulslängen unterschieden wird. Ein 0-Bit wird durch eine kurze High-Phase und eine lange Low-Phase repräsentiert, während ein 1-Bit eine längere High-Phase und eine kürzere Low-Phase aufweist. Jede LED verarbeitet die empfangenen Daten, speichert die Farbwerte und leitet den Rest des Datenstroms an die nächste LED weiter. Die LEDs werden einzeln adressiert, was eine präzise Steuerung der gesamten Kette ermöglicht.[10]

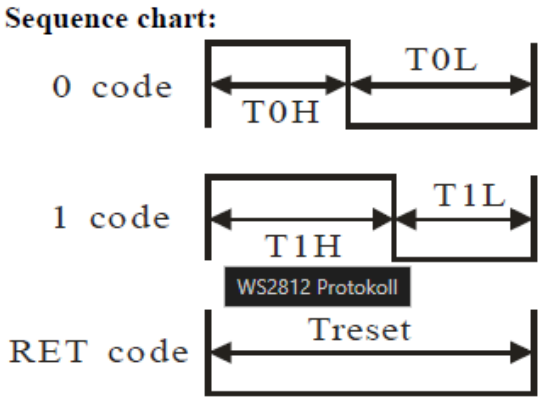


Abbildung 4.2: Neopixel Datenuebertragung.[2]

4.3 HTTP

Das HTTP (Hyper Transfere Protocol) ist das Standardprotokoll für Kommunikation im World Wide Web. Es ermöglicht den Datenaustausch zwischen einem Server und einem Client (z.B Browser). Das Protokoll, welches in der Anwendungsschicht arbeitet, ist zustandslos, sprich jede Anfrage ist unabhängig von vorheriger und nachfolgender Anfrage [11, S. 2].

4.3.1 Funktionsweise

Ein Client, üblicherweise ein Webbrowser, stellt eine Anfrage (Request) an einen Server. Dabei wird eine TCP Verbindung aufgebaut. Der Server antwortet auf die Anfrage mit einer Response, die über einen Status-Code eine erfolgreiche oder fehlerhafte Verbindung signalisiert [11, S. 5]. Es gibt verschiedene Anfragen, wie z.B GET, HEAD, POST, PUT, DELETE. Genutzt wurden die GET, sowie die POST Anfragetypen.

- **GET:** Eine GET_Anfrage wird genutzt, um Daten, wie HTML-Dokumente oder Bilder vom Server anzufordern.
- **POST:** Eine POST_Anfrage ist der GET-Anfrage ähnlich, wird jedoch auch häufig zum Erstellen oder Aktualisieren von Ressourcen verwendet

5 Firmware

Die Firmware ist der Teil des Programms, der direkt mit der Hardware interagiert, grundlegende Funktionen steuert und Schnittstellen zu anderen Systemkomponenten bereitstellt. Zu den Hauptfunktionen (siehe Abbildung 5.1) zählt das Erstellen des Webservers, das Einrichten der WIFI-Verbindung, sowie die I²C-Initialisierung und Kommunikation.

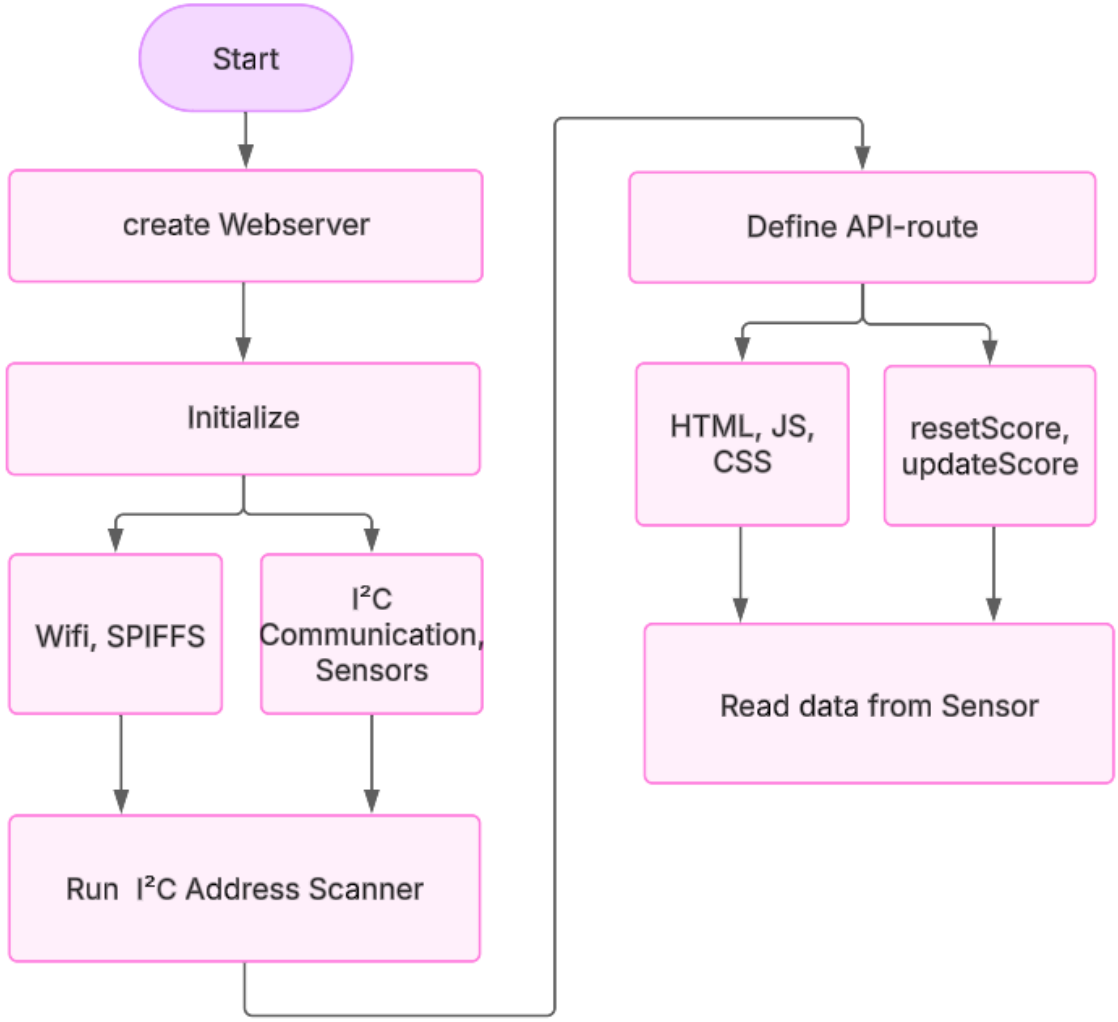


Abbildung 5.1: Programmablauf Firmware

5.1 I²C Kommunikation

I²C (siehe 4.1) dient als Kommunikationsschnittstelle zwischen dem ESP32 und den Hallsensoren. In diesem Zusammenhang übernimmt der Mikrocontroller die Rolle des Masters und steuert die Kommunikation, während die Sensoren als Slaves fungieren. An den I²C-Bus werden seriell mehrere Sensoren angeschlossen, wobei jeder mit einer spezifischen Hardwareadresse angesprochen wird.
Zur Steuerung der I²C-Schnittstelle des ESP32 wird die `Wire.h`[1] Bibliothek verwendet. Diese ist die Standard Arduino-Bibliothek zur Kommunikation über das I²C Protokoll. Sie stellt unter anderem Funktionen wie `write()` und `read()`, zum Senden und Empfangen von Daten bereit. Weitere Funktionen die verwendet wurden, sind `endTransmission()`, `available()`, `beginTransmission()` und `begin()`.

5.1.1 Initialisierung

In der `setup()`-Methode (Codeauschnitt 5.1) wird die Verbindung initialisiert. Dabei wird einerseits die Baudrate angegeben (Z.7), die festlegt, wie viel Symbole pro Sekunde bei einer seriellen Übertragung übertragen werden. Des Weiteren werden die Pins für die Daten -und Taktleitung einerseits definiert (Z. 3,4) und andererseits für die I²C Verbindung festgelegt (Z. 8).

```
1 #include <Wire.h>
2 #include <Arduino.h>
3 #define SDA_PIN 21
4 #define SCL_PIN 22
5
6 void setup() {
7  Serial.begin(115200);
8  Wire.begin(SDA_PIN, SCL_PIN);
9 }
```

Programmcode 5.1: I²C-Initialisierung

[1] https://docs.arduino.cc/language-reference/en/functions/communication/wire/

5.1.2 I²C-Scanner

Der I²C-Adresscanner, siehe Abb. 5.2 bzw. Codeauschnitt 5.2, wird genutzt, um alle am Bus angeschlossenen Geräte zu erkennen. So können einerseits die Hardwareadressen der Geräte identifiziert, andererseits aber auch Fehler in der Hardwareanbindung schnell geortet werden. Die Funktion besteht darin, dass alle möglichen Hardwareadressen von oxoo bis oxFF überprüft werden, um zu detektieren, welche Geräte auf Anfragen antworten. Wird ein Slave am Bus erkannt, so wird dessen Adresse in der Konsole ausgegeben.[12].

Dies wird im nachfolgenden Blockschaltbild schematisch veranschaulicht.

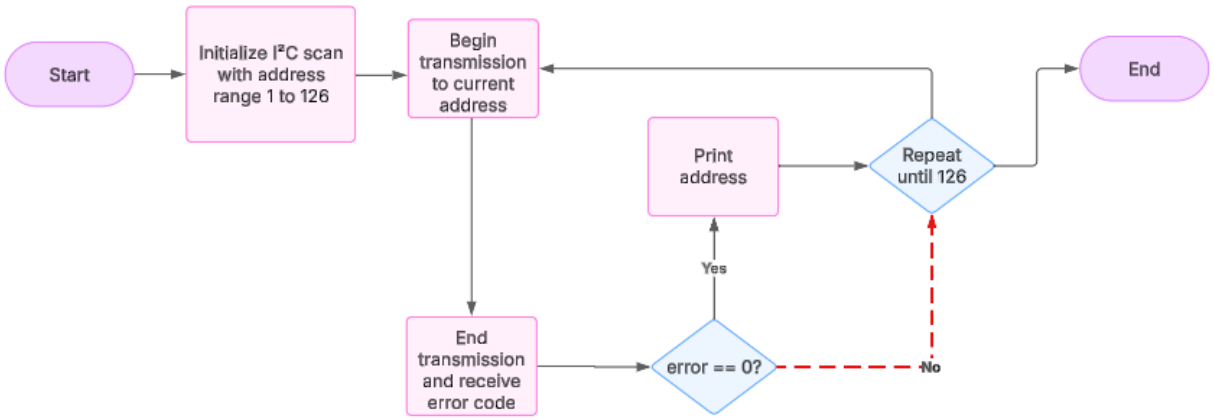


Abbildung 5.2: I²C-Scanner

```
for (address = 1; address < 127; address++)
{
 Wire.beginTransmission(address);
 error = Wire.endTransmission();
 if (error == 0)
 {
  Serial.print("I2C device found at address 0x");
  if (address < 16)
  Serial.print(address, HEX);
 }
}
```

Programmcode 5.2: I²C-Initialisierung

5.1.3 Sleep Mode des Sensors

Um das korrekte Lesen der Sensordaten zu gewährleisten, wird zuerst sichergegangen, dass der Sensor aktiv ist. Dies geschieht durch die Funktion `wakeSensor()`, die vor dem Lesen der Daten aufgerufen wird. Wie im folgenden Ausschnitt aus dem Datenblatt des Sensors (Abb. 5.3) angeführt, wird die Verbindung gestartet, das Register oxC4 ausgewählt und der Wert ox04 in das Register geschrieben, siehe Abbildung 5.4. Dadurch wird das Bit 2 (ob00000100)gesetzt.

ADDR	7	6	5	4	3	2	1	0
0xC0	chipid (RO)				revid (RO)			
0xC1	Dspsigm							
0xC2	Dspsigl							
0xC3						dspsigsel		
0xC4	meas(RO)				Usestore	oneburst	stop	sleep
0xC5								arautoinc

Abbildung 5.3: Sleep_Bit [12](S. 11)



Abbildung 5.4: Wake Sensor

5.1.4 Lesen der Sensordaten

Wie im Datenblatt (Abb. 5.5) angeführt, besteht ein Wert aus 15 Bits. Deshalb ist es notwendig, das Datenwort in zwei Bytes, High- und Lowbyte zu übertragen. Dies ist eine typische Vorgehensweise beim Lesen von Sensordaten mit I²C. Diese zwei Bytes sind im Register oxC1 bzw. oxC2 gespeichert.

ADDR	7	6	5	4	3	2	1	0
0xC0	chipid (RO)				revid (RO)			
0xC1	Dspsigm							
0xC2	Dspsigl							
0xC3						dspsigsel		
0xC4	meas(RO)				Usestore	oneburst	stop	sleep
0xC5								arautoinc
0xC6	sw_low4field	sw_op						

Abbildung 5.5: High Byte/Low Byte Datasheet [12](S. 11)

Mit diesen Informationen folgt das Lesen der Sensordaten nach folgendem Ablauf, siehe Programmauschnitt 5.3. `Wire.begineTransmission()` leitet die Kommunikation ein und übergibt die Hardwareadresse des Sensors, der angesprochen werden soll (Z. 3). Der Master teilt dem Sensor mit, dass aus dem Register reghByte gelesen werden soll (Z. 4). Anschließend wird diese Übertragung beendet, aber ohne Stopbedingung, so dass die Kommunikation aktiv bleibt, und in weiterer Folge Daten gesendet werden können. Der Master fordert mit `Wire.requestFrom()`(Z. 8) genau 1 Byte der gewählten Adresse an. Hier schaltet der Master in den Lesemodus. Nach einer Verzögerung, um sicherzustellen, dass die Daten gesendet sind, bevor sie gelesen werden, wird das Byte in der Variable `highByte` gespeichert (Z. 10). Äquivalent dazu wird auch das Lowbyte gelesen.

```
1  int read_hbyte(uint8_t addr)
2  {
3   Wire.beginTransmission(addr);
4   Wire.write(reg_hByte);
5   Wire.endTransmission(false);
6   delay(10);
7
8   Wire.requestFrom(addr,1);
9   delay(1);
10  byte highByte = Wire.read();
11  delay(10);
12  return highByte;
13 }
```

Programmcode 5.3: Lesen des Highbytes

Anschließend gilt es nach dem Lesen der Daten, die in Form von Low- und High-

byte vorliegen, zusammenzuführen und anschließend in einen handlichen Wert zu konvertieren. Diese Berechnung wird wie im Datenblatt des Sensors beschrieben, durchgeführt.

$$B = (256 \cdot \text{Dpssigm}[6:0] + \text{Dpssigl}[7:0] - 16384) \cdot (0.00125) \tag{5.1}$$

Zuerst sichergestellt, dass sich der Sensor nicht im Sleep-Mode befindet (siehe 5.1.3). Anschließend werden High- und Lowbyte gelesen, gespeichert und zu einem Wert geformt. Als Blockschaltbild verdeutlicht sich dieser Ablauf wie folgt in 5.6.

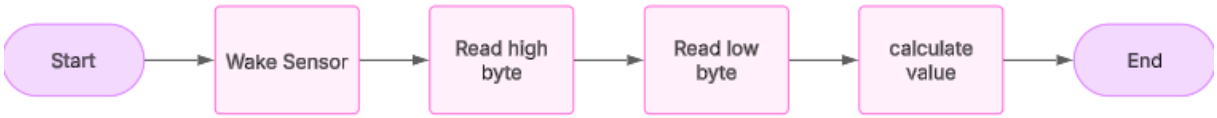


Abbildung 5.6: High Byte/Low Byte [12](S. 11)

Der folgende Programmausschnitt 5.4 zeigt die Umsetzung als Software.

```
1  int get_Data(uint8_t addr){
2   Wire.beginTransmission(addr);
3   Wire.endTransmission();
4   // wake sensor
5   wake_sensor(addr);
6   delay(100);
7   int highByte = read_hbyte(addr);
8   int lowByte = read_lbyte(addr);
9   return ((256 * highByte + lowByte - 16384) * (0.00125));
10 }
11
```

Programmcode 5.4: Lesen des Highbytes

5.2 Webserver

KI-Generiert

Der ESP32 fungiert als Webserver, der über WLAN einen Weboberfläche bereitstellt. Der Webserver wird auf Port 80 gestartet und nutzt die Bibliothek `ESPAsyncWebServer`[a] zur Verarbeitung von HTTP-Anfragen. Dadurch können sowohl statische als auch dynamische Inhalte effizient verwaltet und bereitgestellt werden.

[a] https://docs.arduino.cc/libraries/espasyncwebserver/

5.2.1 Bereitstellung der Daten

Die Bereitstellung der Daten erfolgt über definierte Routen im ESP Webserver. Dabei werden statische Dateien, wie das HTML File aus dem Serial Peripheral Interface Flash File (SPIFFS) geladen, während dynamische Daten, wie der Score nur über JSON-Routen bereitgestellt werden.
SPIFFS ist das Dateisystem des ESP. Es ermöglicht das Speichern und Verwalten von Dateien auf einem Teil des Flash-Speichers des ESP[13]. Die HTML Datei, das CSS File und das Jason Script werden mit SPIFFS gespeichert und anschließend mittels definierter API-Routen dem Client zur Verfügung gestellt. SPIFFS benötigt keine externe Hardware, bietet aber den Vorteil, dass die Dateien dynamisch verwaltet werden können. Die folgende Abbildung 5.7 verdeutlicht den Zusammenhang zwischen SPIFFS, dem ESP und Client.

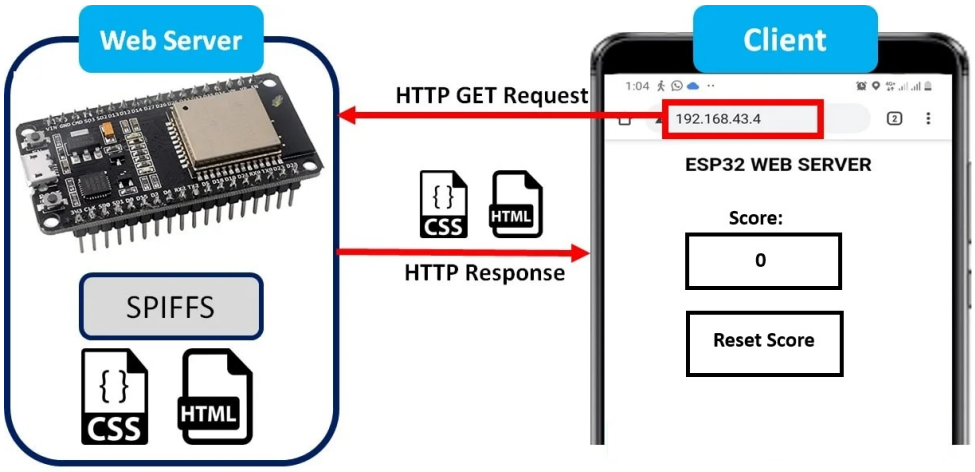


Abbildung 5.7: SPIFFS [14]

Im folgenden Code 5.5 wird angeführt, wie eine neue Route definiert wird und auf eine GET-Anfrage (siehe 4.3) hingewiesen (Z. 1). Daraufhin sucht der ESP im Dateisystem nach `index.html` und sendet diese an den Webbrowser zurück (Z. 2). In weiterer Folge erhält der Webbrowser erhält diese Dateien und zeigt sie entsprechend an.

```
1 server.on("/", HTTP\_GET, [](AsyncWebServerRequest *request) {
2  request->send(SPIFFS, "/index.html", "text/html");
3 });
```

Programmcode 5.5: HTTP_GET Anfrage

Beim Zurücksetzen des Scores, siehe Programmausschnitt 5.6, wird eine POST Anfrage gesendet (Z. 1), sobald auf der Webanwendung der entsprechende Button zum Zurücksetzen betätigt wird. Diese Anfrage wird an die definierte Route `/reset` gesendet. Anschließend wird die Setter-Funktion für den Score aufgerufen (Z. 2), um diesen entsprechend zu aktualisieren. Eine Erfolgsmeldung signalisiert den korrekten Vorgang (Z. 3).

```
1 server.on("/reset", HTTP\_POST, [](AsyncWebServerRequest *request) {
2  setScore(0);
3  request->send(200, "text/plain", "Score zurueckgesetzt");
4 });
```

Programmcode 5.6: HTTP_POST Anfrage

6 Software

Die Software lässt sich in Frontend und Backend unterteilen.

- **Frontend:** Das Frontend ist für die Benutzeroberfläche verantwortlich
- **Backend:** Das Backend regelt die Datenverarbeitung und den Spielablauf.

6.1 Ablauf der Software

Wie im folgenden Blockdiagramm grafisch aufgeschlüsselt läuft auf der Softwareseite des Programms die `Mode1`-Funktion, die für die Programmlogik zuständig ist, während parallel dazu das Aktualisieren der Benutzeroberfläche `fetchData()` alle `2` Sekunden ausgeführt wird.

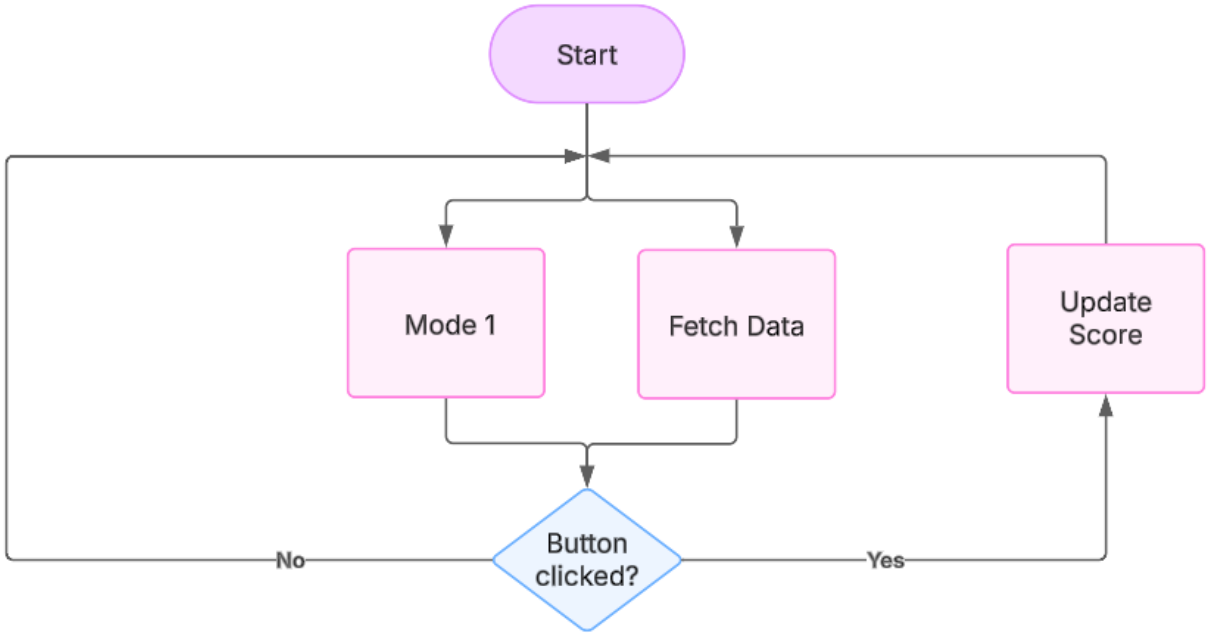


Abbildung 6.1: Ablauf der Software

6.2 Frontend

Das Frontend ist für das Optische der Benutzeroberfläche verantwortlich. Dazu zählen die Webseitenstruktur über HTML, sowie das Design mit CSS.

6.2.1 Hyper Text Markup Language

Hyper Text Markup Language (HTML) ist eine textbasierte Auszeichnungssprache mit der das Grundgerüst, also die Struktur und Inhalte einer Webseite definiert wird, indem Elemente, wie z.B Überschriften, Absätze, Links oder Bilder eingefügt werden. HTML basiert auf sogenannten Tags, die den Webbrowsern mitteilen, in welcher Form Inhalte angezeigt werden sollen[15].
Das HTML File , siehe Codeausschnitt 6.1, besteht aus einer Anzeige für den aktuellen Score, sowie einen Button zum Zurücksetzen des Spielstands. Des Weiteren sind Referenzen zu der entsprechenden CSS- sowie die Js-Datei enthalten, damit der Server weiß, welche Dateien geladen werden sollen.

```
1  <title>Score Anzeige</title>
2  <link rel="stylesheet" href="style.css">
3  </head>
4  <body>
5  <div id="container">
6  <h1>Aktueller Score</h1>
7  <div id="scoreField">Laden...</div>
8  <button id="resetButton">Score zuruecksetzen</button>
9  </div>
10 <script src="script.js"></script>
```

Programmcode 6.1: HTML Struktur

6.2.2 Cascading Style Sheets

Cascading Style Sheets (CSS) ist eine Formatierungssprache, die es ermöglicht, Formatierungen bezüglich Farben, Schriftgröße, Positionierung, sowie Abständen von HTML Dokumenten (Webseiten) zu realisieren[16].

Im folgenden Codeabschnitt 6.2 wird die grafische Gestaltung des Feldes mit CSS, über welches der Score angezeigt wird, erklärt. Dazu werden diverse Parameter wie die Schriftgröße (Z. 2) oder die Breite (Z. 8) definiert.

```
1  #scoreField {
2   font-size: 3rem;              /* Schriftgroesse */
3   font-weight: bold;            /* Textdarstellung */
4   text-align: center;           /* Zentrierung */
5   padding: 20px;                /* Innenabstand*/
6   border-radius: 10px;          /* Ecken des Randes*/
7   width: 250px;                 /* Breite */
8   background-color: #bbdefb;    /* Hintergund*/
9   color: #0d47a1;               /* Schriftfarbe*/
10 }
```

Programmcode 6.2: CSS Styling

6.3 Benutzeroberfläche

Auf der Benutzeroberfläche ist das Feld für die Anzeige des Scores, sowie der Button für die Funktion des Zurücksetzens analog der Strukturierung mit HTML und des Designs mittels CSS zu erkennen.

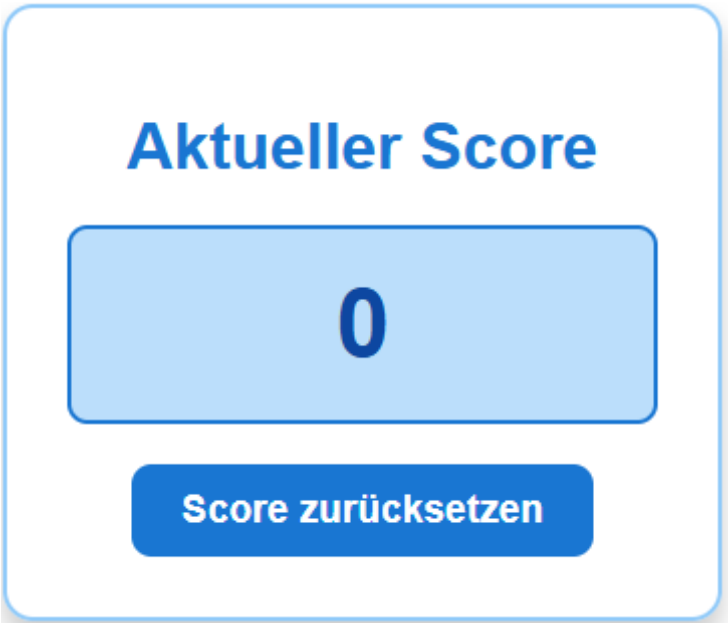


Abbildung 6.2: Webanwendung

6.4 Backend

KI-Inspiriert

Das Backend ist für die Logik und Datenverarbeitung, sowie die Kommunikation zwischen Frontend und Hardware zuständig. In diesen Aufgabenbereich fällt das Empfangen von Anfragen aus dem Frontend und anschließendem Senden entsprechender Antworten. Ein weiterer Bestandteil des Backends die Spiellogik und die damit einhergehende Ansteuerung der Spots.

6.4.1 Kommunikation mit dem Frontend

Die Kommunikation mit dem Frontend erfolgt über HTTP-Anfragen (siehe 4.3), die im JavaScript File verarbeitet werden. Darunter fällt die dynamische Aktualisierung des Scores (siehe Abbildung 6.3), sowie die Logik des Reset-Buttons, (Abbildung 6.4).
So ist in der Methode `fetchScore()`, siehe Abb. 6.3, die regelmäßige Aktualisierung des Scores implementiert. Zuerst wird eine GET-Anfrage an den Server gesendet, um den aktuellen Score zu ermitteln. Anschließend wird der zuletzt angezeigt Score mit dem neu übermittelten Score, um festzustellen, ob sich dieser verändert hat. Ist dies der Fall, so wird der neue Wert im HTML Dokument gespeichert und entsprechend angezeigt. Dieser Wert wird gespeichert, um diesen Vorgang alle 2 Sekunden zu wiederholen.

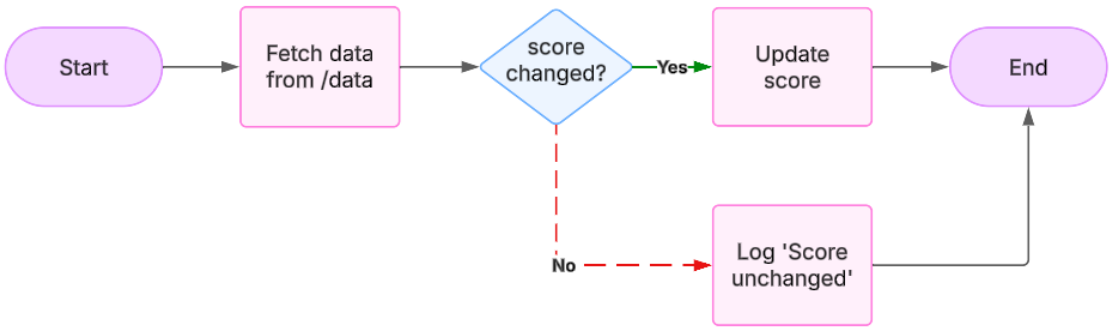


Abbildung 6.3: Dynamische Datenaktualisierung

Die Funktion `resetScore()`, (Abb. 6.4), realisiert die Logik, um mit einem Button auf der Webseite, den Score zurückzusetzen. Zuerst wird mithilfe eines EventListeners gewartet bis der entsprechende Button auf der Webanwendung betätigt wird (Code 6.3). In weiterer Folge wird eine POST-Anfrage, siehe Codeausschnitt 6.4, Z. 1, gesendet und bei einer erfolgreichen Antwort der zuletzt angezeigt Score auf Null gesetzt (Programmausschnitt 6.5, Z. 1). Anschließend wird die Funktion `fetchScore()` aufgerufen, um die Anzeige zu aktualisieren (Code 6.5, Z. 2).

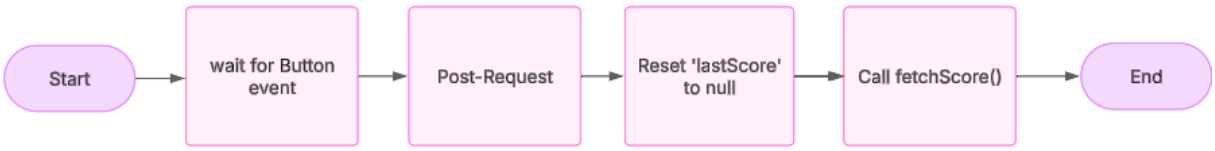


Abbildung 6.4: Reset Score

Event Listener um den Status des Buttons zu überprüfen:

```
1 document.getElementById('resetButton').addEventListener('click',
     resetScore);
```

Programmcode 6.3: Event Listener

Post-Anfrage an den Server:

```
1 const response = await fetch('/reset', { method: 'POST' }); //POST-Rq
2 if (!response.ok) {
3  throw new Error(`HTTP-Error: ${response.status}`);
4 }
```

Programmcode 6.4: Post-Request

Score zurücksetzen und `fetchScore()`:

```
1 lastScore = null;
2 fetchScore();
```

Programmcode 6.5: Reset und fetchScore()

6.4.2 Spiellogik

Die Logik besteht grundsätzlich aus zwei Hauptfunktionen, welche für das Verarbeiten der Daten, das einhergehende Auslesen der Sensoren und Steuern der LEDs verantwortlich sind. Da eine schnelle Datenverarbeitung bzw. Datenabfrage für die Anwendung eine entscheidende Rolle spielt, ist es wichtig, dass vor allem bei den Sensoren immer bekannt ist, welcher gerade aktiv ist. Um dies zu gewährleisten, wurde der Status jedes Sensors in einer vierwertigen Bool-Array gespeichert. So kann statt allen Sensoren, nur der jeweils aktive abgefragt werden, was die Komplexität dieses Vorgangs von O(n) auf O(1) verbessert. Dies bedeutet, dass die Laufzeit unabhängig von der Anzahl der Elemente konstant bleibt und der Vorgang somit effizienter wird. Entsprechend bekommt die Funktion `switchSensor()` , siehe Abbildung 6.5, den Wert des aktives Sensors, also des Spots, an dem die LEDs aktiv sind und vergleicht mit dem Schwellenwert im Ruhezustand. Bei einer gröberen Abweichung, sprich der Sensor erkennt ein starkes Magnetfeld, wird mittels Zufallsgenerator ein neuer aktiver Spot bestimmt und dessen LEDs eingeschalten.

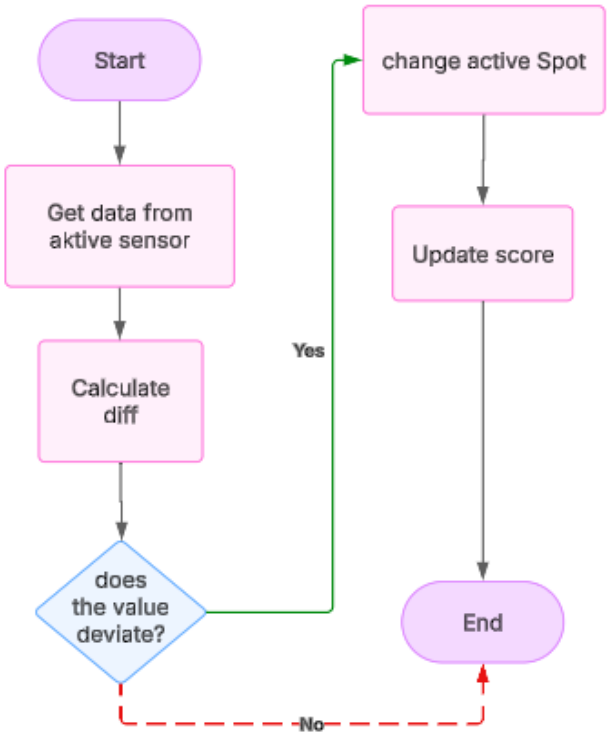


Abbildung 6.5: Switch Sensor

Die Funktion `mode1()`, siehe Abbildung 6.6, überprüft welcher Sensor aktiv ist. Anschließen wird die Funktion `switchSensor()` mit den Übergabeparametern, Index des aktiven Sensors und dessen Hardwareadresse, aufgerufen. In `switchSensor()` werden diese Informationen genutzt, um einerseits den Spot auszuschalten und andererseits sicherzustellen, dass dieser Spot nicht wieder eingeschalten wird.

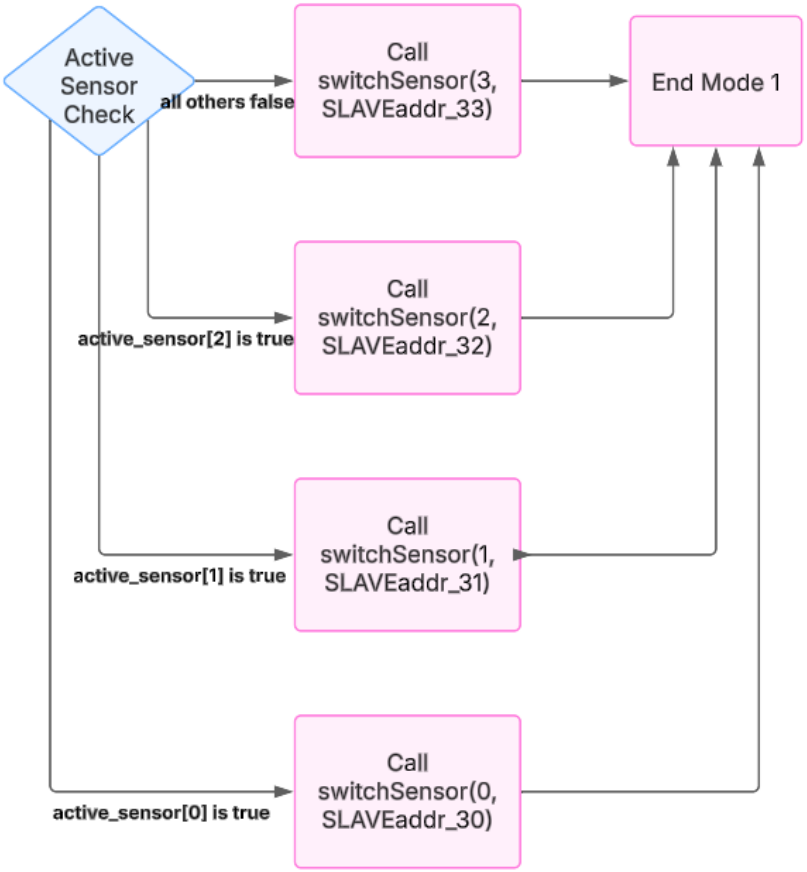


Abbildung 6.6: Spiellogik

6.4.3 LED Ansteuerung

Mithilfe der Adafruit Library [1] werden die LEDs angesteuert (siehe Abschnitt 4.2). Sie werden immer als Vierer-Gruppe behandelt, da auf dem Board jeweils 4 pro Platine einen Spot beleuchten. Im Programmausschnitt 6.6 wird der Data-Pin (Z. 1), für 4 Platinen entsprechend 16 LEDs (Z. 2) und eine Leuchtstärke (0-255) (Z. 3) definiert. Ein Neopixel Objekt mit dem Namen `strip` wird erstellt und die definierten Werte, inklusive Datenrate (`NEO_KHZ800`) und Farbfolge (`NEO_GRB`) übergeben (Z. 4).

[1] https://docs.arduino.cc/libraries/adafruit-gfx-library/

```
#define LED_PIN 18
#define NUM_LEDS 16
#define ledstaerke 150
Adafruit_NeoPixel strip = Adafruit_NeoPixel(NUM_LEDS, LED_PIN, NEO_GRB
  + NEO_KHZ800);
```

Programmcode 6.6: LED Deklaration

Um die LEDs als Gruppe ein und auszuschalten (Abbildung 6.7), wird der jeweilige Spot als Integer-Wert (0 bis 3) übergeben. Mittels Switch-Case wird zwischen den einzelnen Spots unterschieden und anschließend die entsprechenden LEDs eingeschalten. Über die Funktion `strip.color`, welche von der Neopixel Library zur Verfügung gestellt wird, kann eine beliebige RBG Farbe analog der Farbfolge(G-R-B) eingestellt werden. Der Codeausschnitt 6.7 erläutert diesen Sachverhalt. Hier wird der Spot mit dem Index 0 angesprochen und mit der for-Schleife die ersten vier Leuchtdioden der Kette angesprochen, und in der Farbe Grün eingeschalten.

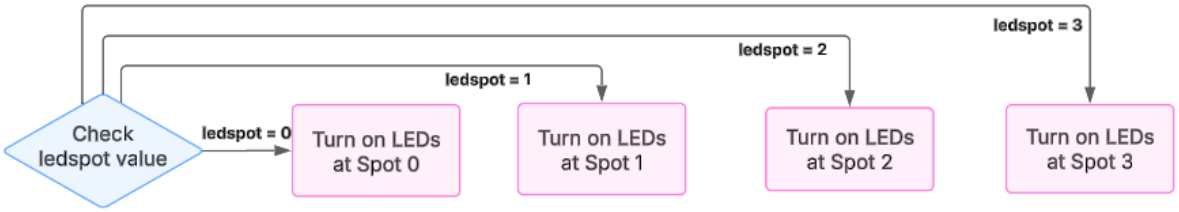


Abbildung 6.7: LED Spot Ansteuerung

```
switch (ledspot) {
 case 0:
 for (int i = 0; i < 4; i++) {
  strip.setPixelColor(i, strip.Color(0,100,0));
  strip.show();
 }
 break;
```

Programmcode 6.7: LED Ansteuerung

Die Methode, die das Ausschalten übernimmt, funktioniert äquivalent zum Einschaltprozess.

7 Fazit und Ausblick

Im Rahmen dieses Projekts konnte die angestrebte Funktionalität im Wesentlichen erfolgreich umgesetzt werden. Die Spieloberfläche ermöglicht eine präzise Bewegung des Pucks, und die Beleuchtung der Spots sowie die Sensorik funktionieren in den meisten Fällen zuverlässig. Die Spiellogik reagiert korrekt auf die erfassten Sensorwerte, sodass ein effektives Trainingssystem bereitgestellt wird.

7.1 Probleme und Lösungen

Mitten im Projekt ergab sich die Schwierigkeit, dass die Hardwareadressen der Sensoren fest gesetzt und nicht, wie zunächst angenommen, softwaretechnisch modifizierbar waren. Aus diesem Grunde mussten zusätzliche Sensoren vom sleben Typ, jedoch mit unterschiedlichen Adressen, bestellt werden. Da jedoch von Herstellerseite nur vier verschiedene Adressvarianten verfügbar sind, beschränkt dies das System auf maximal vier Sensoren.
Eine Lösung für dieses Problem könnte ein I²C-Multiplexer darstellen, der es ermöglicht, mehrere Geräte mit gleicher Hardwareadresse an denselben Bus anzuschließen, indem er die Geräte nacheinander zwischen- bzw. abschaltet. Das würde bedeuten, dass man mehr als vier Sensoren in das System einbauen könnte, und zwar ohne durch die Zahl der fest vergebenen Hardwareadressen eingeschränkt zu sein.
Ein weiteres Problem betrifft die Geschwindigkeit der Sensordatenverarbeitung. In manchen Fällen reagiert die Verbindung nicht schnell genug, sodass der Puck nicht zuverlässig erkannt wird. Dies kann den Spielfluss beeinträchtigen. Eine mögliche Lösung wäre die Verwendung einer Komparatorschaltung, wo direkt auf das Sensorsignal reagiert und die LEDs automatisch umgeschalten werden. Dadurch könnte die Auswertung der Sensordaten durch den ESP32 umgangen werden, was die Latenzen deutlich reduzieren würde.

7.2 Ausblick

Neben Verbesserungsmöglichkeiten der bestehenden Funktionen bietet sich eine Er-

weiterung des Systems an. Sinnvoll wäre z.B. die Integration eines Akkus, um das System unabhängig von einer externen Stromquelle zu machen. Zusätzlich könnten verschiedene Spielmodi implementiert werden, um die Trainingsmöglichkeiten zu erweitern. Beispielsweise könnten Modi für gezieltes Reaktionstraining, Geschwindigkeitsübungen oder Präzisionsaufgaben hinzugefügt werden. Hierbei könnten unterschiedliche Farben und Blinkmuster bei den LEDs unterstützend wirken, da sie eine andere Fokussierung des Trainings erfordern. Eine weitere denkbare Verbesserung wäre die Erweiterung der Sensoranzahl, um ein größeres Spielfeld abzudecken, wozu bereits angesprochene Lösungen genutzt werden könnten.

Abbildungsverzeichnis

1.1 Spieloberflaeche 2
1.2 Blockschaltbild Programmstruktur 3

2.1 Blockdiagramm 4
2.2 Si7210 Hall Effekt Sensor [1] 5
2.3 Pinbelegung Top View [1, S. 21] 6
2.4 WS2812B Gehäuse mit Pinbelegung [2, S. 2] 8
2.5 WS2812B Schaltung [2, S. 5] 9
2.6 WS2812B Absolute Höchstwerte[2, S. 2] 9
2.7 female/male Steckverbinder [3] 10
2.8 Symbol Eagle Steuer- & Messplatine 10
2.9 ESP32-WROOM-32E [4] 11

3.1 Schaltung Prototyp Eagle 14
3.2 Schieberegister 74HC595 [5, S. 31] 15
3.3 SML0805 [6] 16
3.4 Steuerplatine 18
3.6 Messplatine 18
3.5 Steuerplatine 19
3.7 Messplatine Schaltplan 20

4.1 I²C Datenübertragung.[9] 22
4.2 Neopixel Datenuebertragung.[2] 23

5.1 Programmablauf Firmware 25
5.2 I²C-Scanner 27
5.3 Sleep_Bit [12](S. 11) 28
5.4 Wake Sensor 28
5.5 High Byte/Low Byte Datasheet [12](S. 11) 29
5.6 High Byte/Low Byte [12](S. 11) 30
5.7 SPIFFS [14] 31

6.1 Ablauf der Software 33
6.2 Webanwendung 35

6.3 Dynamische Datenaktualisierung 36
6.4 Reset Score 37
6.5 Switch Sensor 38
6.6 Spiellogik 39
6.7 LED Spot Ansteuerung 40

Quellenverzeichnis

[1] *Si7210 I 2 C Hall Effect Magnetic Position and Temperature Sensor Data Sheet*. besucht am 5. März 2025. Adresse: https://www.silabs.com/documents/public/data-sheets/si7210-datasheet.pdf.

[2] *WS2812 Ansteuerung – Mikrocontroller.net*, Mikrocontroller.net, 2025. besucht am 10. Feb. 2025. Adresse: https://www.mikrocontroller.net/articles/WS2812_Ansteuerung?.

[3] *WR-MM Male IDC Connector*. besucht am 4. Feb. 2025. Adresse: https://www.we-online.com/components/products/datasheet/690157001072.pdf.

[4] *ESP32-WROOM-32E*. Adresse: https://eu.mouser.com/datasheet/2/891/esp32_wroom_32e_esp32_wroom_32ue_datasheet_en-1855879.pdf.

[5] *SNx4HC595 8-Bit Shift Registers*, Texas Instruments. besucht am 28. Feb. 2025. Adresse: https://www.ti.com/lit/ds/symlink/sn74hc595.pdf?ts=1738749082740&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252Fde-de%252FSN74HC595.

[6] *SMLo805*, Mouser Electronics. besucht am 3. März 2025. Adresse: https://www.mouser.de/datasheet/2/244/LUMX_S_A0010426821_1-2551978.pdf.

[7] C. Best, *TB2669 I²C Slave Mode*. besucht am 10. Feb. 2025. Adresse: https://ww1.microchip.com/downloads/aemDocuments/documents/OTH/ApplicationNotes/ApplicationNotes/TB2669-I2C-Slave-Mode-00002669B.pdf.

[8] C. Jonischkeit, *Inter-Integrated Circuit (I²C)*. besucht am 10. Feb. 2025. Adresse: https://archive.air.in.tum.de/pub/Main/TeachingWs2014ProseminarMicrocontrollerEm
I2C.pdf.

[9] J. Wu, *A Basic Guide to I²C*. besucht am 10. Feb. 2025. Adresse: https://www.ti.com/lit/an/sbaa565/sbaa565.pdf?ts=1739103088716&ref_url=https%253A%252F%252Fwww.google.com%252F.

[10] *WS2812B Intelligent control LED integrated light source Features and Benefits*. Adresse: https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf.

[11] J. Franke, J. Jaschkowski und A. Miller, *Hypertext Transfer Protocol*, Jan. 2006. besucht am 5. März 2025. Adresse: https://goessner.net/download/learn/mwt/ws2005/presentations/HTTP.pdf.

[12] C. Nelson, *How to Scan and Detect I2C Addresses*. besucht am 18. Feb. 2025. Adresse: https://cdn-learn.adafruit.com/downloads/pdf/scanning-i2c-addresses.pdf.

[13] *File System Storage*. besucht am 25. Feb. 2025. Adresse: http://www.stensat.org/docs/sys395/11_filesystem.pdf.

[14] *ESP32 Web Server with SPIFFS SPI Flash File System*, Microcontrollers Lab, Juli 2023. besucht am 26. März 2025. Adresse: https://microcontrollerslab.com/esp32-web-server-spi-flash-file-system-spiffs/.

[15] L. Campanaro, *HTML einfach erklärt — Web Development*, ONELINE, Feb. 2024. besucht am 19. Feb. 2025. Adresse: https://oneline.ch/blog/html-einfach-erklaert/.

[16] *CSS EINFÜHRUNG Cascading StyleSheets*, 2010. besucht am 19. Feb. 2025. Adresse: https://www.luis.uni-hannover.de/fileadmin/kurse/material/htmlkurs/css-folien_fuer_web.pdf.

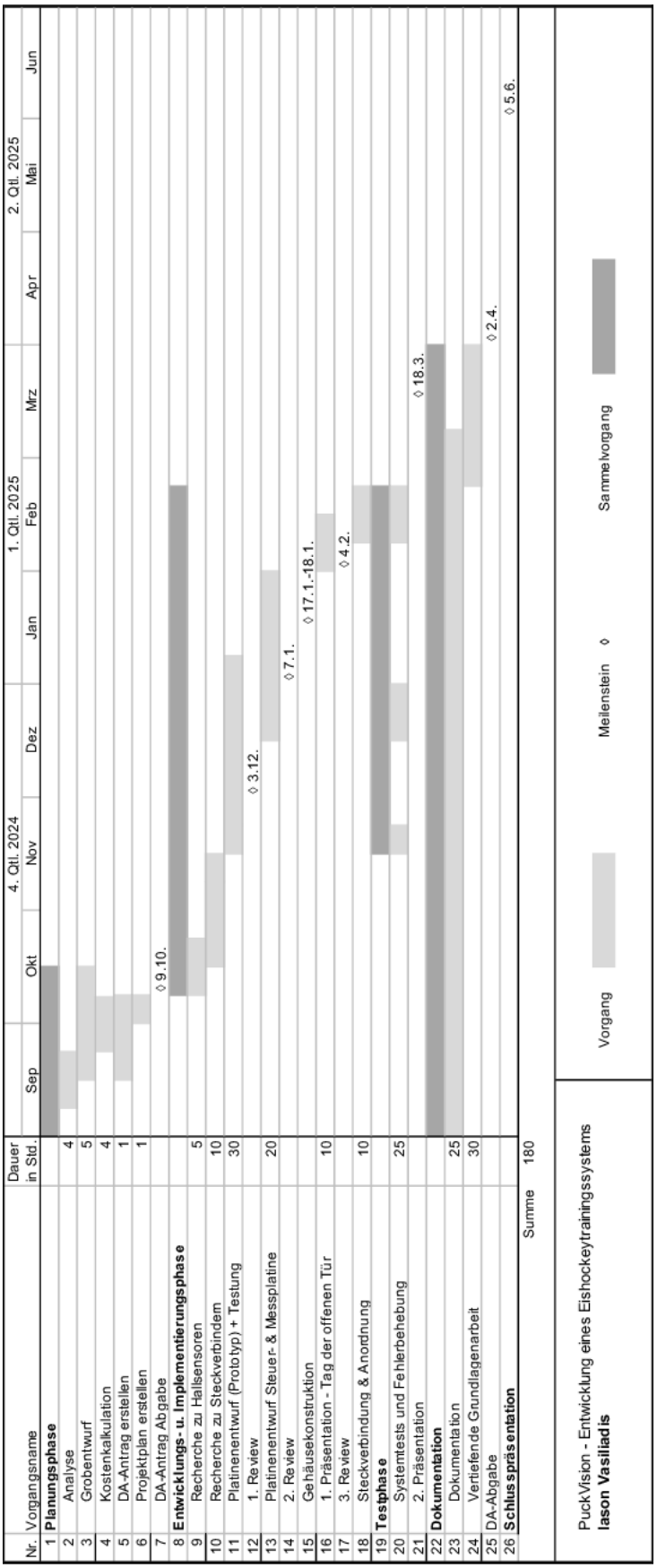
Begleitprotokolle

Name: Iason Vasiliadis

Diplomarbeitstitel: PuckVision - Entwicklung eines Einhockeytrainingssystems

KW	Beschreibung	Zeitaufwand
38	Analyse der Problemstellung	2h
38	Diplomarbeitsantrag erstellen	2h
39	Projektplanung und Aufgabenteilung	1h
39	Evaluieren der Umsetzung	3h
40	Recherche zu Halleffektsensoren	5h
41	Recherche Steckverbinder	5h
42	Recherche LEDs	3h
43	Prototyp Schaltung erstellen	10h
44	Prototyp Board erstellen	4h
45	Löten	5h
46	Fehlerbehebung	8h
47	Eagle libraries erstellen	5h
48	Steuerplatine Schaltung	10h
49	Steuerplatine Board	4h
50	Messplatine Schaltung	10h
51	Messplatine Board	5h
52	Fehlerbehebung	10h
01	Löten	10h
02	Löten	2h
03	Fehlerbehebung	10h
04	Fehlerbehebung	5h
05	Messplatine V2 Schaltung	2h
06	Messplatine V2 Board	5h
07	Löten	5h
08	Spieloberfläche Recherche	3h
09	Spieloberfläche Schaumstoff	2h
10	Spieloberfläche Plexiglas	8h
11	Zusammenfügen der Komponenten	3h
12	Tests	5h
13	Verfassen der Diplomarbeit	10h
14	Verfassen der Diplomarbeit	10h

Gantt-Diagramm



Name: Fabian Gassler

Diplomarbeitstitel: PuckVision - Entwicklung eines Einhockeytrainingssystems

KW	Beschreibung	Zeitaufwand
38	Analyse der Problemstellung	1h
38	Diplomarbeitsantrag erstellen	5h
39	Projektplanung und Aufgabenteilung	1h
39	Evaluieren der Umsetzung	5h
40	Recherche zu Halleffektsensoren	3h
41	Recherche I2C	10h
42	Implementierung der Testsoftware	10h
43	Implementierung der Testsoftware	4h
44	Fehlerbehebung	8h
45	Fehlerbehebung	5h
46	Implementierung der I2C Verbindung	8h
47	Implementierung der I2C Verbindung	8h
48	Fehleranalyse	5h
49	Recherche zu I2C	3h
50	Fehlerbehebung	10h
51	Fehlerbehebung	5h
52	Codeoptimierung	4h
01	Implementierung der LED Ansteuerung	7h
02	Implementierung der Spiellogik	10h
03	Implementierung der Spiellogik	4h
04	Optimieren der Software	5h
05	Recherche zu Webserver	5h
06	Implementieren des Webservers	10h
07	Implementieren des Webservers	2h
08	Optimieren der Software	5h
09	Optimieren der Software	10h
10	Optimieren Weboberfläche	5h
11	Optimieren Weboberfläche	3h
12	Verfassen der Diplomarbeit	2h
13	Verfassen der Diplomarbeit	10h
14	Verfassen der Diplomarbeit	10h

Gantt-Diagramm

